vate/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html

容 近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

第 25 章：HTTPX、可靠重试与同步 I/O 边界

本章的核心结论是：**异步 HTTP 的迁移单位不是某一行 `await`，而是从取得连接到释放资源的整条调用链。**

一个 Agent tool 可能依次等待连接池、传输请求、解析 JSON、校验模型，遇到临时故障后退避重试，最后还要查询数据库。只把函数声明改成 `async def`，不会自动让同步库变成非阻塞，也不会自动获得总超时、可靠重试和资源清理。

先把一次可靠调用记成下面这条主线：

```
请求作用域创建 deadline、AsyncClient 和并发许可
        ↓
等待连接池 → 发送/接收 HTTP → 检查状态 → 解析 JSON → 校验模型 → 判断业务结果
        │
        ├─ 可恢复且允许重复：释放本次传输许可 → 在剩余预算内退避 → 再尝试
        ├─ 不可恢复或不能重复：保留原始原因并立即失败
        └─ 取消：向上传播；随后按所有权关闭 response、client 和其他资源

同步 SDK / sqlite3：把完整阻塞工作单元交给线程，但线程仍有自己的生命周期
```

本章默认只访问 `127.0.0.1` 或 `MockTransport`，不需要 API Key、付费服务或公网。

学习目标

完成本章后，你应该能：

- 说清 `await client.get(...)` 与普通 `response.json()` 为什么不同；
- 为多个 tool 共享一个 `AsyncClient`，利用连接池并在所属作用域退出时关闭；
- 区分 connect/read/write/pool timeout、整体 deadline、Semaphore 和连接池容量；
- 分层处理传输、HTTP 状态、JSON、模型和业务失败；
- 用 `client.stream(...)` 与 `aiter_lines()` 消费 JSONL，并在提前退出时关闭响应；
- 设计最大次数、退避、jitter、`Retry-After` 和剩余预算都明确的只读重试；
- 解释并发许可在一次传输后释放还是跨退避持有的影响；
- 用 `asyncio.to_thread()` 适配同步函数，并让 SQLite 连接在线程内创建和关闭；

README
ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

- 用 `asyncio.to_thread()` 适配同步函数，并让 SQLite 连接在线程内创建和关闭；
- 解释为什么取消等待 `to_thread()` 的 Task 不等于停止已经运行的线程。

25.1 先建立 Java → Python 心智模型

可以先用熟悉的 Java 组件定位职责，但不能只按 API 名字做一一替换：

Java 中熟悉的概念	Python 中的对应做法	不能直接等同的边界
JDK `HttpClient`、Spring `WebClient`	`httpx.AsyncClient`	`AsyncClient` 的发送和关闭都可能需要 `await`；它必须在事件循环中使用
`CompletionStage` / `CompletableFuture`	coroutine 经 `await` 得到结果，或包装成 `asyncio.Task` 并发运行	调用 `async def` 只创建 coroutine，不会立刻调度；`await` 也不等于新建线
`try-with-resources` + `AutoCloseable`	`async with` + 异步上下文管理器	Python 的退出阶段可以异步等待；拥有者退出作用域时才负责关闭，借用者不应越权关闭
HTTP 客户端连接池	`AsyncClient` 内部连接池和 `httpx.Limits`	连接数、Task 并发数和每秒请求数是三种不同容量，不能用一个参数代替
客户端 connect/read timeout	HTTPX 的 connect/read/write/pool timeout	分阶段 timeout 不是整个 tool 的 SLA；多次尝试与退避还要共享一个绝对 dead
Resilience4j Retry / Spring Retry	显式重试状态机、退避、jitter、`Retry-After`	只有“失败可恢复”且“操作允许重复”才可重试；`CancelledError` 不是可恢复异常
Jackson + Bean Validation	`response.json()` + Pydantic `model_validate()`	类型提示不会自动校验网络输入；Pydar 是运行时边界校验，不是 Java 编译期类型检查
`ExecutorService.submit(syncCall)`	`await asyncio.to_thread(sync_call, ...)`	取消等待方不会强制停止已运行线程；厄 timeout、副作用和关闭仍由同步代码负
JDBC 工作单元持有自己的连接	在线程函数内创建、使用并关闭	SQLite 默认连接受创建线程约束；

.DME × README × +
r/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

JDBC 工作单元持有自己的连接	在线程函数内创建、使用并关闭 `sqlite3.Connection`	SQLite 默认连接受创建线程约束；不能在事件循环线程建连接后随意跨线程

最重要的调度差异是：一个事件循环线程会交替推进多个 Task。当前 Task 只有执行到一个尚未完成的 **awaitable** 时，才把执行机会交回事件循环。 `await` 表示"这里允许暂停等待"，不是"这里一定启动线程"，也不是"里面调用的所有函数都会自动异步化"。

本章用到的最小前置

实际存在的第 12 章已经建立了"请求构造 → 传输 → HTTP 状态 → JSON → 数据/业务"的分层；第 21 章已经介绍取消和绝对截止时间，第 22 章介绍 TaskGroup，第 23、24 章还分别讨论并发许可、连接池容量、流和异步资源所有权。本章不跨章导入代码，只在真实 I/O 场景中复用下面四条结论：

- `asyncio.Semaphore(n)` 限制同时进入某段代码的 Task 数； `async with semaphore` 在异常和取消时也会释放许可。它不是 QPS 限流器。
- `loop.time()` 是事件循环的单调时钟； `deadline = loop.time() + budget` 只计算一次，后续每层都从这个绝对时刻计算剩余预算。
- `async with` 适合进入和退出都可能等待的资源；谁创建资源，谁应明确负责最终关闭。
- `async for` 是逐项异步等待，不自动并行；消费流中途 `break` 时，仍需由响应上下文确定性关闭底层连接。

如果没有读过前置章节，也可以继续阅读；上面四条会在真实 I/O 场景中再次展开。

25.2 HTTP 调用究竟在哪里异步等待

哪些操作需要 `await`

最小正确用法：

```
import httpx


async def fetch_payload(base_url: str) -> object:
    async with httpx.AsyncClient(base_url=base_url) as client:
        response = await client.get("/document")
        response.raise_for_status()
```

OME × README × +
ffolders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

哪些操作需要 `await`

最小正确用法：

```
import httpx


async def fetch_payload(base_url: str) -> object:
    async with httpx.AsyncClient(base_url=base_url) as client:
        response = await client.get("/document")
        response.raise_for_status()
        return response.json()
```

判断标准不是“它和 HTTP 有关”，而是该 API 是否返回 awaitable：

操作	是否 `await`	原因
`client.get/post/request(...)`	是	等待连接、发送与接收是异步 I/O
`client.aclose()`	是	异步关闭客户端资源
`response.aread()`	是	流模式中异步读取余下正文
`response.aiter_lines()`	用 `async for`	每次取得下一行可能要等网络数据
`response.raise_for_status()`	否	检查内存中的状态码
`response.json()`	否	对已缓冲正文做同步 JSON 解析
`Document.model_validate(payload)`	否	在当前线程做模型校验

普通 `client.get()` 默认在返回前读取响应正文，所以随后 `response.json()` 不需要网络等待。流模式刻意不预读完整正文，必须用 `aread()` 或异步迭代方法取得内容。不要凭函数名字猜 `await`；查签名、类型提示和官方文档。

如果习惯 Java 的 `CompletableFuture`，可以把 coroutine 暂时理解为“尚未执行到结果的异步计算描述”，但两者不能完全等同。调用 `client.get(...)` 得到 coroutine 后，既没有阻塞等待，也没有因为调用表达式本身就新建线程；当前 coroutine 被 `await`，或被包装成 Task 交给事件循环后，调用链才会推进。

错误反例：

ADME × README × +
ar/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

错误反例：

```
response = client.get(url)  # 错：这里只得到 coroutine，不是 Response
payload = await response.json()  # 错：json() 返回普通 Python 值，不是 awaitable
```

客户端是一段有所有者的长生命周期资源

`AsyncClient` 保存连接池、默认 headers/cookies、认证和配置。典型 Agent 应在一次应用、请求作用域或明确的服务组件中创建它，再传给多个 tool：

```
async def fetch_pair(base_url: str) -> tuple[int, int]:
    async with httpx.AsyncClient(base_url=base_url) as client:
        first = await client.get("/documents/1")
        second = await client.get("/documents/2")
        # return 前会先执行 __aexit__，异步关闭 client。
        return first.status_code, second.status_code
```

工具函数只借用 client，不应在一次调用结束时关闭调用者仍要复用的池。官方文档特别提醒不要在高频循环中每次创建一个客户端：那会不断创建新池，失去 keep-alive、DNS/TLS/连接复用的价值。

这与 Java 服务中复用 `HttpClient`、`WebClient` 或连接池的工程原则相同：长生命周期组件通常由应用容器或请求作用域拥有，业务方法只借用。区别是 `AsyncClient.aclose()` 本身也要异步等待，所以 Python 用 `async with` 表达生命周期；普通 `try-with-resources` 的同步 `close()` 不能完整描述这一点。

```
# 教学反例：循环每轮都新建并关闭一个池。
for item in items:
    async with httpx.AsyncClient() as client:
        await client.get(url, params={"item": item})
```

本地 fixture 使用 HTTP/1.1、准确 `Content-Length` 并记录客户端端口。完整示例连续发两次请求，服务端观察到相同端口，证明**这一次受控运行**复用了 TCP 连接。连接池只提供复用机会；服务端、代理、协议错误、空闲过期或提前停止流都可能让下一次请求新建连接，不能把"端口永远相同"写成业务合同。

如果不能使用 `async with`，必须在所有路径最终执行 `await client.aclose()`。全局 client 并非天然错误，但必须有对应的应用 shutdown 钩子；"模块导入时创建、从不关闭"才是问题。

四种传输 timeout、整体 deadline 和容量限制

本章配置：

```
timeout = httpx.Timeout(connect=0.20, read=0.08, write=0.20, pool=0.03)
limits = httpx.Limits(max_connections=1, max_keepalive_connections=1)
```

HTTPX 的四个 timeout 都是特定阶段的等待限制：

限制	控制什么	典型异常	不控制什么
connect	建立 socket 连接的等待	`ConnectTimeout`	后续读正文、全部重试总时长
read	等下一块响应数据的等待	`ReadTimeout`	完整下载的总时长、JSON 校验
write	发送下一块请求数据的等待	`WriteTimeout`	服务端处理总时长
pool	从客户端连接池取得连接的等待	`PoolTimeout`	创建多少 Task、外部服务 QPS

`Limits.max_connections` 决定池允许的并发连接总数；`max_keepalive_connections` 决定可保留的空闲 keep-alive 连接数。示例让一个慢请求占住唯一连接，再发第二个请求，稳定观察 `PoolTimeout`。这验证了本机 HTTPX/httpcore 的池行为，不是 `MockTransport` 的结果。

四类机制不要混为一个"超时/限流"：

机制	放在哪里	回答的问题
HTTPX timeout	客户端/单请求	一个传输阶段最多空等多久?
`asyncio.timeout_at(deadline)`	整个 tool/请求作用域	尝试、退避和编排合计还能等多久?
`asyncio.Semaphore`	应用定义的临界范围	同时允许多少个 tool 进入这段操作?
HTTPX `Limits`	HTTP 连接池	同时建立/保留多少 HTTP 连接?

Semaphore 可以包含"构造请求到拿到响应"的一次尝试，也可以只包 socket 请求，范围由你的容量含义决定。它不限制每秒请求数；需要 QPS 限制时还要单独的速率算法。整体 deadline 建立在协作取消上，也不是硬中断：同步 JSON/模型校验若长时间占住事件循环，超时回调同样不能在中间强行抢占。

五层错误应该保留五层证据

README × README +
ate/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

五层错误应该保留五层证据

类型提示只给静态检查器看，不会主动检查网络数据。Pydantic 模型把"不可信 JSON"转换成进入业务逻辑前的稳定对象：

```
from pydantic import BaseModel, ConfigDict, Field


class Document(BaseModel):
    model_config = ConfigDict(strict=True, extra="forbid", frozen=True)
    id: int
    title: str
    score: float = Field(ge=0.0, le=1.0)
```

`strict=True` 拒绝把字符串 `"25"` 悄悄转成整数，`extra="forbid"` 拒绝未声明字段，`Field` 再检查分数范围。这些规则在 `model_validate()` 被调用时运行，不是 Python 编译器提供的静态类型保证。

本章的 `fetch_document()` 使用以下顺序：

```
await client.get()
  ├─ httpx.TransportError ──────────→ TransportFailure
  ▼
response.raise_for_status()
  ├─ httpx.HTTPStatusError ─────────→ HttpStatusFailure
  ▼
response.json()
  ├─ json.JSONDecodeError ──────────→ ResponseJsonFailure
  ▼
Document.model_validate(payload)
  ├─ pydantic.ValidationError ──────→ ResponseValidationFailure
  ▼
Document
```

应用异常用 `raise ... from error` 保留 `__cause__`。调用方可以按稳定的 tool 异常处理，日志仍能看到 HTTPX/Pydantic 的原始证据。

这类似 Java 用领域异常包装底层异常并把原异常作为 cause，但不要只保留一条新消息。传输失败和合同失败的恢复策略不同；若全部压成同一个空对象或模糊异常，上层重试器就失去了分类依据。

对接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

若响应还有 `{"code": "QUOTA_EXCEEDED"}` 之类业务码，应在模型结构通过后再判断，形成第五层业务失败。HTTP 200、合法 JSON 和模型正确都不能替它证明业务成功。反过来，HTTP 404 即使带合法 JSON，也仍是 HTTP 层失败。

不要这样写：

```
try:
    return (await client.get(url)).json()
except Exception:
    return {}
```

空结果、连接失败、404、非法 JSON 和代码缺陷全被压成 `{}`，调用者既不能恢复也无法排障。取消虽然不属于 `Exception`，宽泛吞错仍会隐藏其他程序缺陷。

JSONL：行是协议边界，chunk 不是

本章本地端点返回 `application/x-ndjson`，每一行都是一个完整 JSON 值：

```
{"type":"token","text":"你好"}\n
{"type":"done","usage":1}\n
```

正确消费结构：

```
import json

import httpx


async def read_first_event(client: httpx.AsyncClient) -> object:
    async with client.stream("GET", "/events") as response:
        response.raise_for_status()
        async for line in response.aiter_lines():
            return json.loads(line)
    raise ValueError("流中没有事件")
```

TCP 分段和 HTTP chunk 由传输层决定：一个 chunk 可能只有半行，也可能含多行。因此下面的写法错误：

```
async for chunk in response.aiter_bytes():
```

ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html

的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

TCP 分段和 HTTP chunk 由传输层决定：一个 chunk 可能只有半行，也可能含多行。因此下面的写法错误：

```
async for chunk in response.aiter_bytes():
    event = json.loads(chunk)  # 错：任意网络 chunk 不是 JSON 文档边界
```

如果协议是 JSONL，用 `aiter_lines()`；如果是 SSE，要按 SSE 字段和空行规则解析；如果是一个超大 JSON 数组，需要真正的增量 JSON 解析器，不能擅自把它当 JSONL。

示例读取第一行后 `break`。`async for` 的退出本身不代表任意底层资源都立即释放，但外层 `async with client.stream(...)` 离开时会关闭 `Response`。运行断言观察到上下文内 `is_closed == False`、退出后 `is_closed == True`，然后才放行服务端尾部并等待服务线程结束。

真实本地服务与 `MockTransport` 各证明什么

手段	本章用它验证	不能据此声称
回环 HTTP 服务	TCP 传输、HTTP/1.1 复用、read/pool timeout、流提前关闭	公网 DNS/TLS/代理、真实服务延迟与副作用
`MockTransport`	精确制造连接异常、状态、坏 JSON、坏模型和重试序列	DNS、socket、连接池或网络 timeout 真会怎样

`MockTransport` 的 handler 是进程内替身；HTTPX timeout 配置不会把任意同步 mock handler 自动变成真实网络阶段。单元测试用替身覆盖状态机，集成测试再覆盖真实传输，两者不能互相冒充。

25.3 怎样设计不会放大故障的重试

先判断“失败可恢复吗”，再判断“操作能重复吗”

重试是再次执行，不是异常消失器。两个条件都成立才进入自动重试：

1. 失败可能是暂时的；
2. 重复该操作不会产生不可接受的第二次副作用。

本章只重试按合同只读的 GET：

失败	默认策略	原因

ME × README × +
olders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
2.60
刚接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

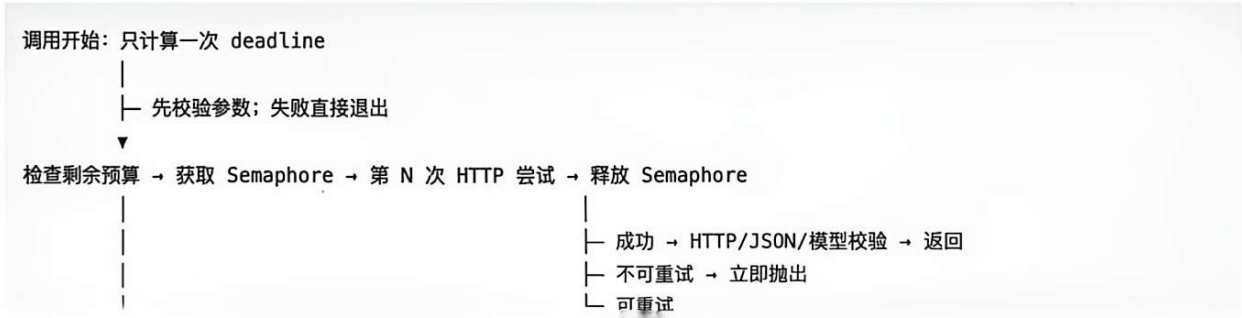
本章只重试按合同只读的 GET：

失败	默认策略	原因
`ConnectError/ConnectTimeout`	可考虑重试	连接阶段可能暂时失败；仍受预算限制
`ReadError/ReadTimeout`	只读操作可考虑	服务端可能慢；写操作此时可能已经执行
`WriteError/WriteTimeout`	只读请求可谨慎考虑	已发送多少数据可能不确定
`RemoteProtocolError`	可有限重试	连接/协议可能暂时损坏
HTTP 429	尊重 `Retry-After` 后考虑	服务端明确限流
HTTP 502/503/504	可有限重试	网关/服务常是暂时不可用
HTTP 400/401/403/404	不自动重试	参数、认证、权限或不存在通常不会自行变好
JSON/模型校验错误	不自动重试	合同不匹配；盲重试常只复制同一错误
tool 业务失败	按业务合同	“余额不足”“禁止操作”不是传输失败
`CancelledError`	立即传播	用户/上层已要求停止，不是重试信号

`PoolTimeout` 在本章不列入自动重试：它首先表示本地池容量/持有时间问题。立刻重新排队可能加剧拥塞，应先由上层背压、并发限制或容量配置解决。真实系统可按自己的负载合同调整，但要明确理由。

如果熟悉 Resilience4j 或 Spring Retry，可以把下面的实现看作显式展开的 retry policy。但 Python 中写一个装饰器并不能省掉策略设计：装饰器仍必须知道哪些异常可恢复、调用是否允许重复、取消如何传播，以及每次等待还剩多少总预算。

一次有限重试怎样运行



一次有限重试怎样运行

```
调用开始：只计算一次 deadline
        │
        ├─ 先校验参数；失败直接退出
        ▼
检查剩余预算 → 获取 Semaphore → 第 N 次 HTTP 尝试 → 释放 Semaphore
        │                                          │
        │                                          ├─ 成功 → HTTP/JSON/模型校验 → 返回
        │                                          ├─ 不可重试 → 立即抛出
        │                                          └─ 可重试
        │
        ├─ 已达 max_attempts → RetryExhausted
        ├─ 退避大于剩余预算 → RetryBudgetExceeded
        └─ 可取消地 await sleep(delay) → 回到剩余预算检查
```

`max_attempts=3` 表示**总共最多发三次**，不是"第一次加三次重试"。这类命名能减少 off-by-one 错误。每一次失败都记录 attempt，最终错误保留最后失败类别。

指数退避的一个简单形式：

```
base = policy.base_delay * 2 ** (failed_attempt - 1)
jitter = base * policy.jitter_ratio * randomizer.random()
client_delay = base + jitter
delay = min(max(client_delay, server_retry_after), policy.max_delay)
```

- 退避让失败流量不立即压回服务端；
- jitter 让大量客户端不在同一毫秒再次齐射；
- `Retry-After` 可以是整数秒或 HTTP-date，本章两种都解析；
- `max_delay` 是本地等待上限。本章会截断更长提示；生产系统也可选择直接失败/排队，而不是比服务端提示更早重发；
- 测试传入固定种子的 `random.Random(25)`，业务代码不应依赖某个具体随机值。

HTTP 的 `Retry-After` 与方法幂等语义见：

- RFC 9110：Retry-After
- RFC 9110：Idempotent Methods

一个 deadline 管全部尝试和退避

ОМЕ × README × +
folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

一个 deadline 管全部尝试和退避

错误写法：

```
for attempt in range(3):
    async with asyncio.timeout(2):  # 每轮重新获得完整 2 秒
        await one_attempt()
```

三次尝试可能消耗近 6 秒，退避还在预算外。正确做法是在入口计算一次：

```
loop = asyncio.get_running_loop()
deadline = loop.time() + 2.0
result = await fetch_read_only_with_retry(..., deadline=deadline)
```

发送前、失败后和退避前都重新计算 `deadline - loop.time()` 。 `asyncio.timeout_at(deadline)` 覆盖实际等待；若服务端要求退避 2 秒而只剩 0.05 秒，示例直接抛 `RetryBudgetExceeded` ，不会先睡满再发现来不及。

`await asyncio.sleep(delay)` 是取消点。完整示例等到"已经进入退避"的 Event，再取消 Task，稳定观察 `CancelledError` ；不会靠恰好在某个 sleep 毫秒内击中任务。

退避时要不要持有并发许可

本章选择"许可只覆盖一次网络尝试"：

```
async with limiter:
    response = await client.get(...)
# 已释放许可
await asyncio.sleep(delay)
```

这样一个上游故障不会让所有许可都被退避中的请求占满。示例使用容量 1：重试 Task 进入十秒退避后，另一个协程仍能立即获取许可，再取消重试 Task。这是释放时机的事件和状态证据。

另一种选择是把许可代表"整个逻辑调用槽"，让一次调用从首次尝试到最终完成始终占槽。它能更强地限制活跃调用总数，但退避会降低吞吐并可能饿死新请求。没有普遍答案；先给许可命名：它限制的是"在途网络传输"还是"完整工具调用"，再让代码范围匹配名字。

为什么不能对 POST/写 tool 盲目重试

ɔME × README × +
folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

为什么不能对 POST/写 tool 盲目重试

read timeout 只说明客户端没有及时读到响应，不说明服务端没有执行。付款、发消息、创建任务、扣 LLM 配额或数据库提交可能已经完成；再次 POST 会产生第二次副作用。

写操作要自动重试，至少要有一项可靠依据：

- 协议和服务端合同明确操作幂等；
- 客户端提供稳定 idempotency key，服务端真正保存并去重结果；
- 有事务状态查询，可先确认上一请求是否提交；
- 有业务级补偿，并接受它不是原子撤销。

HTTP 方法名只能提供通用语义线索，最终以具体 API 合同为准。即使 GET，若服务端实际偷偷写审计/计费状态，也应把这些影响纳入策略；即使 POST，带可靠幂等键的创建接口也可能允许安全重放。本章用只读工具，是为了把传输重试和副作用问题分开。

25.4 已有同步库怎样接入事件循环

`async def` 不会改造里面的同步调用

事件循环一次只执行一个 Task 的 Python 代码。Task 只有在 `await` 一个尚未完成的 awaitable 时，才把控制权交回循环。下面的 `requests.get()` 是普通同步函数：

```
async def wrong(url: str) -> object:
    return requests.get(url, timeout=1)  # 阻塞事件循环线程
```

函数外面的 `async def` 不会改变 `requests`、同步 SDK、`sqlite3` 或 `time.sleep` 的内部实现。阻塞期间，其他 HTTP tool、取消回调、heartbeat 和 timeout 回调都不能在这个事件循环线程运行。

本章的确定性实验让本地服务端等待一个 heartbeat Event：

- 直接调用 `requests.get()` 时，已创建的 heartbeat Task 无法运行，服务端保险等待到期并返回 `heartbeat_seen=False`；
- `await asyncio.to_thread(requests.get, ...)` 时，事件循环恢复调度，heartbeat 及时设置事件，服务端返回 `heartbeat_seen=True`。

`to_thread` 接受的是同步 callable 和参数

```
import asyncio

import requests


def sync_get_json(url: str) -> object:
    with requests.get(url, timeout=(0.2, 1.0)) as response:
        response.raise_for_status()
        return response.json()


async def fetch_json(url: str) -> object:
    return await asyncio.to_thread(sync_get_json, url)
```

第一项是函数对象，后面是传给它的位置/关键字参数。下面的反例先在事件循环里执行调用，再把结果传给 `to_thread`，已经来不及：

```
# 错：requests.get(...) 在进入 to_thread 之前就执行了。
response = await asyncio.to_thread(requests.get(url))
```

`to_thread` 适合把有清晰边界、会阻塞的同步 I/O 迁移出事件循环。它不是把整个旧代码库随手线程化的理由：线程池容量、库的线程安全、事务所有权、底层 timeout、日志上下文和进程退出都仍需设计。

它最接近 Java 中把阻塞调用提交给 `ExecutorService`，但取消语义更弱：取消等待 `to_thread()` 的 asyncio Task，不会像 `Future.cancel(true)` 那样尝试中断工作线程。即使 Java 侧收到 interrupt，底层库也可能忽略它；Python 这里则根本不会向任意运行中线程注入 `CancelledError`。

SQLite：把完整工作单元和连接放在同一线程

下面的最小查询返回与连接、cursor 脱钩的普通值；本章项目再把每行转换成不可变 dataclass：

```
import asyncio
import sqlite3
from contextlib import closing
from pathlib import Path
```

```python
import asyncio
import sqlite3
from contextlib import closing
from pathlib import Path


def query_database(path: Path, minimum_score: float) -> list[tuple[int, str, float]]:
    with closing(sqlite3.connect(path, timeout=0.5)) as connection:
        rows = connection.execute(
            "SELECT id, title, score FROM documents WHERE score >= ? ORDER BY score DESC",
            (minimum_score,),
        ).fetchall()
    return [(int(row[0]), str(row[1]), float(row[2])) for row in rows]


async def load_documents(path: Path) -> list[tuple[int, str, float]]:
    return await asyncio.to_thread(query_database, path, 0.90)
```

关键不是只把 `execute()` 放线程，而是：在线程函数内创建连接、绑定参数、执行、读取结果、转换成不依赖 cursor 的普通对象、关闭 cursor/连接，然后才把结果返回事件循环。

默认 `sqlite3.connect(..., check_same_thread=True)` 会拒绝在另一个线程使用创建出来的连接。不要在事件循环线程创建默认连接，再把它随意传给工作线程；也不要为绕过异常直接设置 `check_same_thread=False`，却不给并发写入和事务加明确串行化。

可以把它类比为一个 JDBC 工作单元：获得连接、绑定参数、执行、物化结果和关闭资源应由同一工作单元完成。这里比常见 JDBC 代码多一条 Python/SQLite 限制——默认连接还记录了创建线程，所以"主线程建连接、线程池执行 SQL"并不是安全的机械翻译。

另一个易错点是：`with connection:` 管理提交/回滚，**不会关闭连接**。本章用 `contextlib.closing()` 明确调用 `close()`；初始化写入则嵌套 `with connection:` 管事务。官方资料：sqlite3 连接上下文管理器 和 `contextlib.closing`。

内存数据库 `":memory:"` 通常属于单条连接；若每个线程各开新连接，它们不会自动看到同一份普通内存库。本章使用临时文件数据库，让初始化连接关闭后，查询线程的新连接仍能读到固定数据，离开 `TemporaryDirectory` 后再删除。

异步驱动还是线程适配

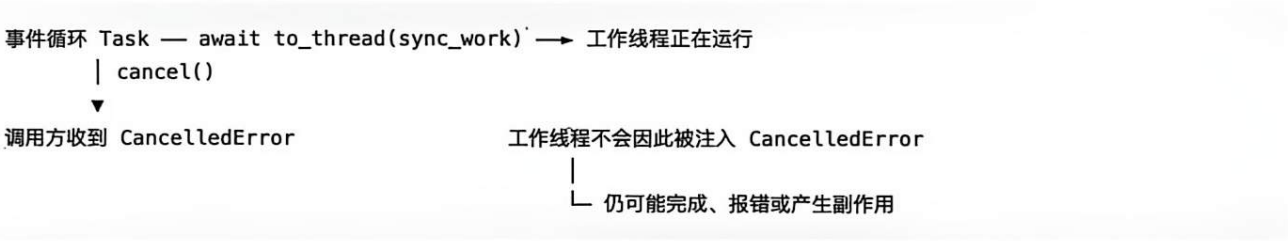
README × README +
ate/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

异步驱动还是线程适配

情况	倾向选择	要继续核对
客户端有成熟原生异步 API	异步客户端/驱动	协议取消、池、事务、流和 shutdown 语义
现有同步 SDK 稳定且调用量有限	`to_thread` 适配	线程安全、线程池、底层 timeout、关闭方式
SQLite 小型/遗留查询	完整查询放 `to_thread`	锁等待、事务边界、每线程连接
大量高并发数据库请求	驱动自己的 async 池	池容量、服务端取消、连接回收和背压
CPU 密集转换	单独评估	进程、原生扩展、自由线程构建或算法优化

"API 名字带 async"不代表底层都一样。网络数据库驱动可能直接把非阻塞 socket 接入事件循环；aiosqlite 官方说明则明确写出每条连接使用一个共享线程和请求队列。选择 API 时看它的实现、取消和资源合同，不要只看函数是否写了 `await` 。

取消 `to_thread` 等待，不等于停止线程



完整示例先用 Event 确认工作线程已经启动，再取消等待它的 Task。此时 `Task` 已取消，但 `finished.is_set()` 仍是 `False` ；放行底层线程后，列表中出现"线程中的底层工作仍然完成"。

这带来三个责任：

- 同步 HTTP/SDK 自己仍需有限 connect/read timeout；
- 数据库锁等待、事务和查询本身要有底层限制或可协作停止机制；
- 组件退出时要知道仍运行的线程由谁观察，不能把已取消 Task 当成工作已消失。

Python 不能安全地从外部强行终止任意正在执行的线程。若同步库支持线程安全的 cancel/close 句柄，可以设计显式协作；否则只能停止等待、记录未知结果，并依赖底层 timeout 最终结束。远端副作用与 HTTP 写重试一样，需要幂等、查询状态或补偿机制。

的主要价值是避免 I/O 阻塞事件循环，不要表现它普遍提供 CPU 并行加速。CPU 任务

`to_thread` 的主要价值是避免 I/O 阻塞事件循环，不要声称它普遍提供 CPU 并行加速。CPU 任务是否并行取决于解释器构建、代码是否释放 GIL、原生库和执行策略；应单独测量后选择进程池、专用执行器、原生向量库或算法优化。

25.5 一次 Agent 请求怎样组合这些边界

`combined` 入口做一件小而完整的事：

1. 入口只计算一次 `deadline`；
2. 创建一个 `AsyncClient` 和一个 Semaphore；
3. 在同一 `asyncio.timeout_at(deadline)` 与 TaskGroup 中，同时启动只读 HTTP tool 和 SQLite 线程查询；
4. HTTP tool 接受共享 client、limiter 和 deadline；本次本地服务正常，因此一次尝试成功；
5. 成功退出 TaskGroup 后读取结果；
6. 依次退出客户端上下文和临时目录，数据库连接已在线程函数内部关闭。

```
请求作用域（deadline）
├─ AsyncClient（连接池所有者）
├─ Semaphore（网络尝试容量）
└─ TaskGroup
   ├─ HTTP GET → 分层校验 → Document
   └─ to_thread → 在线程内 open/query/close SQLite
```

真实 Agent 还要决定：某个 tool 失败时是否取消兄弟、哪些业务失败允许部分成功、线程工作是否能在请求结束后安全留存。这些是 TaskGroup/业务编排策略，不应埋进通用 HTTP 重试函数。

25.6 Java 工程师最容易带过来的误判

反例	后果	修正
高频循环内新建 `AsyncClient`	连接池无法有效复用	在更外层创建并传入多个 tool
忘记关闭 client/stream	连接和文件描述符泄漏	`async with` 或所有路径 `aclose()`
把任意 bytes chunk 当 JSON	半个/多个 JSON 导致随机解析失败	按 JSONL/SSE 等明确协议边界解析
把 read timeout 当总 deadline	重试和处理时间无限膨胀	外层传同一个绝对 deadline
Semaphore 当 QPS 限流器	只限在途数，不限单位时间请求数	单独设计速率限制

README × README × +
e/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

对所有 `TransportError` 无限重试	放大故障、延迟无上界	白名单、最大 attempts、预算和退避
退避时无意持有所有许可	新请求全部饿死	明确许可语义和 `async with` 范围
超时后自动重发所有 POST	远端副作用可能重复	幂等合同/idempotency key/状态查询
`async def` 中直接 `requests.get`	事件循环、取消、其他 tool 全被堵住	原生 async client 或 `to_thread`
`to_thread(sync_call())`	同步调用已在事件循环执行	传函数对象和参数
跨线程共享默认 sqlite 连接	`ProgrammingError` 或状态竞争	每个完整工作单元在线程内建/关连接
认为取消 Task 会杀死线程	底层仍可执行和产生副作用	底层 timeout、协作停止、明确善后

提交前逐项问：

1. client、stream、数据库连接和线程工作的所有者分别是谁？
2. 传输 timeout、整体 deadline、并发上限和池容量各自在哪里？
3. 哪些失败会重试，为什么既“可能恢复”又“允许重复”？
4. attempt 包含哪些阶段，退避是否消耗同一个预算，是否可取消？
5. 取消以后，远端请求、事务或线程中还有什么可能继续？

25.7 递进练习

练习不给完整答案。建议复制本章函数到自己的草稿文件，先写预期事件，再运行。

练习 1：增加完整 JSONL 消费

要求：给 fixture 的流增加 `tool-start`、`tool-result`、`done` 三种事件，让消费者自然读到 `done` 才结束；为每种事件定义最小字段校验。

输入：固定 tool 名 `search_docs`，固定 result `{"count": 2}`。

验收标准：

- 使用 `client.stream()`、`aiter_lines()` 和每行一次 `json.loads()`；
- 不解析任意 `aiter_bytes()` chunk；
- 缺 `type` 或字段类型错误时抛模型错误，不伪装成流自然结束；

README × README +
e/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

验收标准：

- 使用 `client.stream()` 、 `aiter_lines()` 和每行一次 `json.loads()` ；
- 不解析任意 `aiter_bytes()` chunk；
- 缺 `type` 或字段类型错误时抛模型错误，不伪装成流自然结束；
- 正常结束和读取一项后提前退出，两条路径的 response 都最终 `is_closed` 。

练习 2：把单次尝试预算加入重试

要求：在共同 tool deadline 内，再为每次 HTTP 尝试设置更短的 attempt deadline；不要让每轮获得一份新的 tool 总预算。

输入：tool 预算 0.30 秒、单次尝试最多 0.08 秒、最多 3 次、固定退避 0.02 秒。

验收标准：

- 三个绝对时刻都用 `loop.time()` 体系计算；
- attempt deadline 是 `min(tool_deadline, now + 0.08)` ；
- 单次 read timeout 与 attempt timeout 能在日志中区分；
- 用户外部取消仍是 `CancelledError` ，不会被改写成 retry/budget 失败。

练习 3：设计写操作的幂等重试

要求：为“创建任务”POST 设计一个本地服务端去重表；客户端对每个逻辑创建生成一次固定 idempotency key，重试时复用，不得每次生成新 key。

输入：任务标题 `"分析文档"` ；服务端第一次已经创建记录但故意断开响应，第二次返回原记录。

验收标准：

- 两次网络 attempt 携带同一 key；
- 服务端最终只有一条记录；
- 客户端能区分“首次创建”和“去重后取回”；
- README 明确：只有服务端真正实现去重时，key 才能保证该合同。

练习 4：线程中的 SQLite 取消标记

要求：给一个分批读取函数传入 `threading.Event` 停止标记；每批之间检查标记并尽快结束。Task 取消后由所有者设置标记，再等待线程的 `finished` Event。

输入：固定 100 行数据，每批 10 行；取消前用 Event 确认至少开始一批。

练习 4：线程中的 SQLite 取消标记

要求：给一个分批读取函数传入 `threading.Event` 停止标记；每批之间检查标记并尽快结束。Task 取消后由所有者设置标记，再等待线程的 `finished` Event。

输入：固定 100 行数据，每批 10 行；取消前用 Event 确认至少开始一批。

验收标准：

- 连接只在线程函数内创建和关闭；
- 不依赖固定 sleep 决定何时取消；
- 能分别观察 Task 已取消、线程看到停止标记、连接已关闭；
- 说明这是合作式停止，不能替代数据库自己的 query/lock timeout。

练习 5：为容量做一次审计

要求：同时启动 20 个只读 tool，应用 Semaphore 为 3，HTTPX `max_connections` 为 2；记录等待应用许可、等待池、正在传输、退避四种状态的数量。

输入：第 1～5 个请求第一次返回 503，其余直接成功；固定 jitter 随机源。

验收标准：

- 网络尝试并发从未超过 3，真实连接从未超过 2；
- 能解释为什么 Semaphore=3 仍可能出现 pool 等待；
- 退避不持有网络尝试许可；
- 所有任务共用调用方给定的 deadline，取消整组后没有未回收 Task。

25.8 小结、验证边界与最后自测

把本章压缩成四句话：

1. `AsyncClient` 是带连接池的长生命周期资源，创建者负责关闭，tool 只借用。
2. 一次调用要同时区分传输 timeout、整体 deadline、Task 并发和连接池容量。
3. 重试必须同时满足“错误可能恢复”和“操作允许重复”，并消耗同一份总预算。
4. `to_thread()` 只把阻塞等待移出事件循环，既不会把同步库变成原生异步库，也不会让 Task 取消自动停止工作线程。

本章在 CPython 3.14.7、HTTPX 0.28.1、Pydantic 2.13.5、Requests 2.34.2 上运行完整入口。

README × README +
vate/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
容 近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

本章在 CPython 3.14.7、HTTPX 0.28.1、Pydantic 2.13.5、Requests 2.34.2 上运行完整入口。
运行时断言验证了：

- 两个真实本地 HTTP 请求在本次运行中复用同一客户端端口；
- 事件已确认服务端收到请求后，客户端分别触发 `ReadTimeout` 和 `PoolTimeout`；
- JSONL 第一行按行解析，提前退出 stream 后 `Response` 已关闭；
- MockTransport 精确产生传输、HTTP、JSON、Pydantic 四层错误及 cause；
- 暂时失败在第 3 次成功，一直 503 精确尝试 3 次，400 只尝试 1 次；
- `Retry-After` 超出剩余预算时不再等待，退避中取消传播 `CancelledError`；
- 退避期间唯一 Semaphore 许可可被其他协程获取；
- 直接 Requests 阻塞 heartbeat，而 `to_thread` 版本不阻塞；
- SQLite 固定查询返回预期两行，连接在线程函数中关闭；
- 取消等待 `to_thread` 的 Task 后，已运行线程仍可完成底层工作；
- 组合示例退出后关闭客户端、数据库连接、本地服务和临时目录。

静态检查另行验证格式、常见代码问题和类型一致性；它们不等于上述运行验证。

没有验证的范围：公网 DNS、TLS、代理、HTTP/2、多地址连接、真实网络 write timeout；真实 LLM、MCP 和数据库驱动的取消协议；生产连接池/QPS 容量；POST 幂等键和远端副作用；进程关闭时长时间线程的治理。MockTransport 尤其不能证明这些真实传输行为。

最后请不看代码口述：

1. 一次请求中谁创建和关闭 `AsyncClient`，tool 为什么不关闭借来的 client?
2. connect/read/write/pool timeout、deadline、Semaphore、Limits 各控制什么?
3. 一次失败为什么会或不会重试，最大尝试次数和剩余预算怎样变化?
4. 消费 JSONL 提前停止时，哪层上下文保证 response 关闭?
5. 取消等待 `to_thread` 后，为什么线程和远端副作用仍可能继续?

能沿着"资源所有权 → 一次尝试 → 分层校验 → 退避/剩余预算 → 取消后底层状态"完整回答，才算真正跨过了可控 asyncio 实验与真实 I/O 的边界。

25.9 运行本章示例

仓库根 `pyproject.toml` 要求 Python `>=3.14`，Ruff 和 Pyright 也以 3.14 为目标。本章使用以下文件：

文件	职责

25.9 运行本章示例

仓库根 `pyproject.toml` 要求 Python `>=3.14`，Ruff 和 Pyright 也以 3.14 为目标。本章使用以下文件：

文件	职责
`README.md`	独立教程、选择标准、反例、练习与验证边界
`example.py`	HTTP、重试、同步 I/O 三组实验和组合入口
`local_http_fixture.py`	自动启动和关闭的本地 HTTP/1.1 服务
`requirements.txt`	HTTPX、Pydantic、Requests 的固定运行版本

从仓库根目录安装并检查依赖：

```
.venv/bin/python -V
.venv/bin/python -m pip install -r java_to_python/chapters/25_async_http_io/requirements.txt
.venv/bin/python -m pip check
```

可以先分主题运行，再运行完整入口：

```
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py http
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py retry
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py sync
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py combined
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py all
```

运行时重点观察：同一个 client 是否复用连接，read timeout 与 pool timeout 是否分别出现，JSONL 提前退出后 response 是否关闭，退避时 Semaphore 是否已释放，以及取消 Task 后工作线程是否仍能完成。`-B` 只是不生成 `.pyc`；现有 `run_all.py` 只支持前 10 章，本章没有接入它。

当前环境已经安装开发工具时，还可以执行：

```
.venv/bin/ruff check java_to_python/chapters/25_async_http_io
.venv/bin/ruff format --check java_to_python/chapters/25_async_http_io
source .venv/bin/activate
pyright java_to_python/chapters/25_async_http_io/example.py \
```

README × README +
vate/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8855af9f4dfc4e2d2c4aebf4f48.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

从仓库根目录安装开发和测试依赖：

```
.venv/bin/python -V
.venv/bin/python -m pip install -r java_to_python/chapters/25_async_http_io/requirements.txt
.venv/bin/python -m pip check
```

可以先分主题运行，再运行完整入口：

```
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py http
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py retry
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py sync
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py combined
.venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py all
```

运行时重点观察：同一个 client 是否复用连接，read timeout 与 pool timeout 是否分别出现，JSONL 提前退出后 response 是否关闭，退避时 Semaphore 是否已释放，以及取消 Task 后工作线程是否仍能完成。`-B` 只是不生成 `.pyc`；现有 `run_all.py` 只支持前 10 章，本章没有接入它。

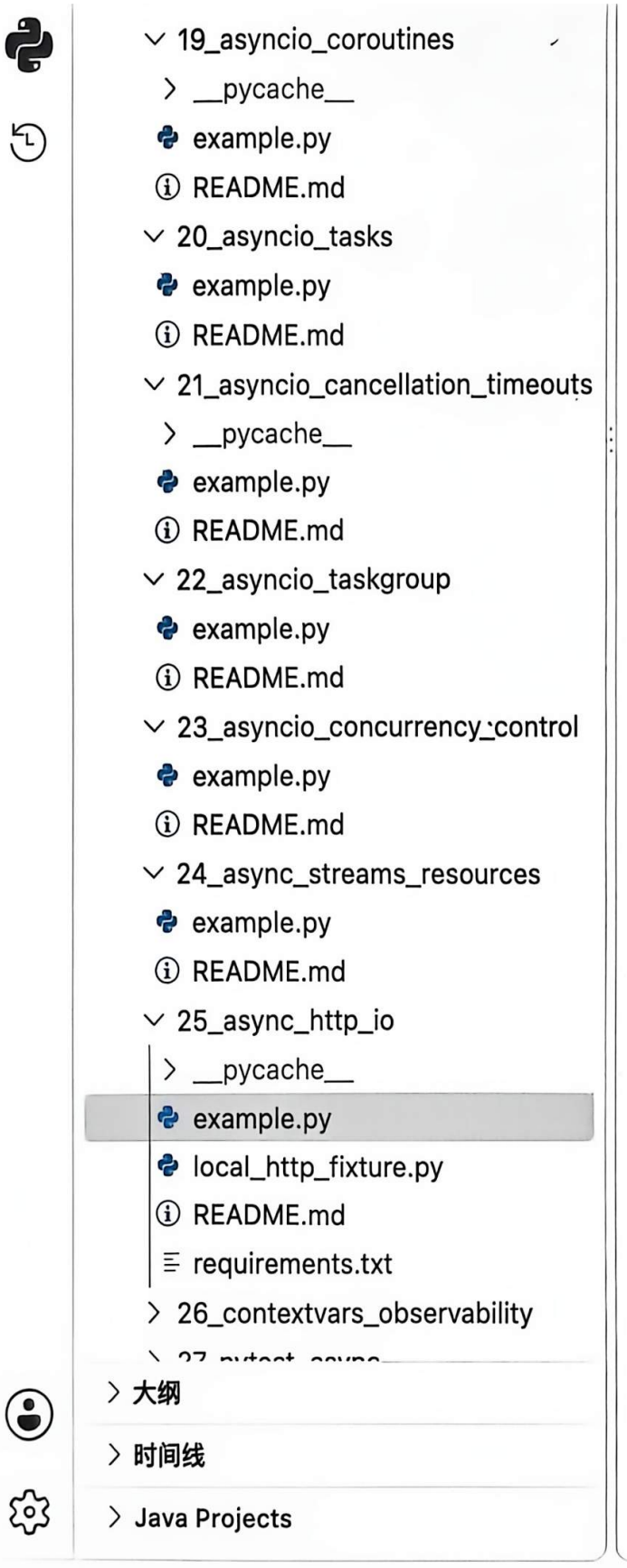
当前环境已经安装开发工具时，还可以执行：

```
.venv/bin/ruff check java_to_python/chapters/25_async_http_io
.venv/bin/ruff format --check java_to_python/chapters/25_async_http_io
source .venv/bin/activate
pyright java_to_python/chapters/25_async_http_io/example.py \
  java_to_python/chapters/25_async_http_io/local_http_fixture.py
```

本章实测环境为 CPython 3.14.7、HTTPX 0.28.1、Pydantic 2.13.5、Requests 2.34.2、Ruff 0.16.5 和 Pyright 1.1.411。HTTPX 0.28.1 的包元数据要求 Python 3.8+，但这次本地运行只能证明本项目在该环境中的兼容性，不能替上游承诺所有 Python 3.14 环境。

继续核对 API 合同时，优先阅读官方资料：

- HTTPX 异步支持与客户端生命周期
- HTTPX timeout与连接池容量
- HTTPX 异常层级与自定义 transport
- Pydantic 模型与 `model_validate()`
- Python 3.14 `asyncio.to_thread()`
- Python 3.14 `sqlite3.connect()`




```
11  # 数据主线是：HTTP 响应 -> I
12  # 本章所有公网 URL 都由 Mock
13
14  # 标准库分别提供：命令行入口、i
15  # inspect 只用于在运行时展示“
16  import argparse
17  import asyncio
18  import inspect
19  import json
20  import random
21  import sqlite3
22  import tempfile
23  import threading
24  from collections.abc import S
25  from contextlib import closin
26  from dataclasses import datac
27  from datetime import UTC, date
28  from email.utils import parsed
29  from http import HTTPStatus
30  from pathlib import Path
31
32  # HTTPX 是原生异步 HTTP 客户端；Re
33  # local_http_fixture 是同目录模块
34  # Pydantic 在 JSON 解码后执行运行时
35  import httpx
36  import requests
37  from local_http_fixture import
```

```
"""第 25 章：HTTPX、可靠请求与同步 I/O 边界。

从仓库根目录运行：
    .venv/bin/python -B java_to_python/chapters/25_async_http_io/example.py all

也可把最后的 all 换成 http、retry、sync 或 combined，逐节观察。
"""

# 建议按文件中的定义顺序阅读：先认识 HTTP/模型合同和重试策略，再看三个独立实验，
# 最后看 combined 如何把 HTTP 协程与同步数据库查询放进同一个请求预算。
# 数据主线是：HTTP 响应 -> Python JSON 值 -> Pydantic Document；失败则转换成稳定的应用异常。
# 本章所有公网 URL 都由 MockTransport 接管，真实 socket 只访问配套 fixture 的 127.0.0.1。

# 标准库分别提供：命令行入口、asyncio 编排、JSON/SQLite 同步处理、线程协调和临时目录。
# inspect 只用于在运行时展示“返回协程的方法”和“普通方法”的区别。
import argparse
import asyncio
import inspect
import json
import random
import sqlite3
import tempfile
import threading
from collections.abc import Sequence
from contextlib import closing
from dataclasses import dataclass
from datetime import UTC, datetime
from email.utils import parsedate_to_datetime
from http import HTTPStatus
from pathlib import Path

# HTTPX 是原生异步 HTTP 客户端；Requests 刻意作为同步客户端反例。
# local_http_fixture 是同目录模块，负责启动可控的本地 HTTP/1.1 服务。
# Pydantic 在 JSON 解码后执行运行时模型校验；它和静态类型提示承担不同职责。
import httpx
import requests
from local_http_fixture import FixtureState, LocalHttpFixture
```

```python
import requests
from local_http_fixture import FixtureState, LocalHttpFixture
from pydantic import BaseModel, ConfigDict, Field, ValidationError

# HTTPX 的 timeout 不是一个笼统总时限，而是连接、等待下一块响应数据、写入和等待连接池
# 四个阶段各自允许空等多久。真正覆盖“多次尝试 + 退避”的总预算稍后由 deadline 表示。
HTTP_TIMEOUT = httpx.Timeout(connect=0.20, read=0.08, write=0.20, pool=0.03)

# 池最多只有一条连接并保留一条空闲 keep-alive 连接，便于实验稳定展示连接复用和池等待超时。
# 这限制的是 HTTP 连接数量，不等于 asyncio Task 数量，也不是每秒请求数限制。
HTTP_LIMITS = httpx.Limits(max_connections=1, max_keepalive_connections=1)

# frozenset 是不可变集合：成员测试仍是 `code in RETRYABLE_STATUS_CODES`，但策略不会被意外增删。
# 这里只把通常可能短暂恢复的限流/网关错误加入白名单；400 等参数错误不应盲目重发。
RETRYABLE_STATUS_CODES: frozenset[int] = frozenset({429, 502, 503, 504})

# 圆括号构造一个异常类元组，稍后可直接写 `except RETRYABLE_TRANSPORT_ERRORS as error`。
# 有意不包含 PoolTimeout：池耗尽首先是本地容量/背压问题，立即重试往往只会重新排队。
RETRYABLE_TRANSPORT_ERRORS: tuple[type[ConnectError], type[Connect... = (
    httpx.ConnectError,
    httpx.ConnectTimeout,
    httpx.ReadError,
    httpx.ReadTimeout,
    httpx.WriteError,
    httpx.WriteTimeout,
    httpx.RemoteProtocolError,
)


# BaseModel 类似带运行时校验能力的 DTO。HTTP JSON 先是 dict/list 等普通 Python 值，
# 只有通过 `Document.model_validate(payload)` 后，调用方才拿到字段合同稳定的 Document 实例。
class Document(BaseModel):
    """进入 Agent 内部前的稳定数据形状。"""

    # strict=True 禁止把字符串 "25" 悄悄转成整数 25；extra="forbid" 拒绝未声明字段；
    # frozen=True 禁止正常方式重新给模型字段赋值。它们是 Pydantic 的运行时规则，
    # 不能简单等同于 Java 编译期类型检查——也不意味着对象内部任意嵌套值都会深度不可变。
```

```python
# BaseModel 类似带运行时校验能力的 DTO。HTTP JSON 先是 dict/list 等普通 Python 值，
# 只有通过 `Document.model_validate(payload)` 后，调用方才拿到字段合同稳定的 Document 实例。
class Document(BaseModel):
    """进入 Agent 内部前的稳定数据形状。"""

    # strict=True 禁止把字符串 "25" 悄悄转成整数 25；extra="forbid" 拒绝未声明字段；
    # frozen=True 禁止正常方式重新给模型字段赋值。它们是 Pydantic 的运行时规则，
    # 不能简单等同于 Java 编译期类型检查，也不意味着对象内部任意嵌套值都会深度不可变。
    model_config = ConfigDict(strict=True, extra="forbid", frozen=True)

    id: int
    title: str
    # Field 给 score 追加闭区间约束；类型正确但小于 0 或大于 1 也会校验失败。
    score: float = Field(ge=0.0, le=1.0)


# 下面先建立应用层 I/O 异常树。调用方可以统一捕获 ToolIOError，也可以按阶段精细恢复；
# 原始 HTTPX、JSON 或 Pydantic 异常会通过 `raise ... from error` 保存在 `__cause__` 中。
class ToolIOError(RuntimeError):
    """本章应用层 I/O 异常的共同基类。"""


class TransportFailure(ToolIOError):
    """连接、读写或协议传输失败。"""


class HttpStatusFailure(ToolIOError):
    """收到了响应，但 HTTP 状态不成功。"""


class ResponseJsonFailure(ToolIOError):
    """成功 HTTP 响应的正文不是合法 JSON。"""


class ResponseValidationFailure(ToolIOError):
    """JSON 合法，但不符合模型合同。"""
```

```python

class ResponseValidationFailure(ToolIOError):
    """JSON 合法，但不符合模型合同。"""


class RetryExhausted(ToolIOError):
    """可重试失败用完最大尝试次数。"""


# 总预算耗尽单独继承内置 TimeoutError，而不伪装成某次 HTTP I/O 失败；
# 上层若对全局超时有统一清理策略，可以和其他 TimeoutError 一起捕获。
class RetryBudgetExceeded(TimeoutError):
    """重试调用共享的绝对截止时间已经耗尽。"""


# `@dataclass` 会根据字段生成初始化、比较和展示方法，可类比减少 Java record/POJO 样板代码，
# 但字段注解默认仍不会像 Java 编译器那样阻止错误类型传入。
# frozen=True 禁止字段重新赋值，slots=True 则为实例声明固定槽位，不再给每个实例创建 __dict__。
@dataclass(frozen=True, slots=True)
class RetryPolicy:
    # max_attempts 包含第一次请求：默认总共最多发送 3 次，并非“首次 + 3 次重试”。
    max_attempts: int = 3
    # base_delay 是首次退避基数，随后按 2 的幂增长；max_delay 是单次退避的本地上限。
    base_delay: float = 0.01
    max_delay: float = 0.10
    # jitter_ratio 控制在指数退避上最多额外增加多大比例的正向随机抖动。
    jitter_ratio: float = 0.20

    # dataclass 生成的 __init__ 完成字段赋值后会自动调用 __post_init__。
    # frozen 只限制赋值，不会自动校验数值范围，所以不变量仍需在这里显式检查。
    def __post_init__(self) -> None:
        if self.max_attempts < 1:
            raise ValueError("max_attempts 必须至少为 1")
        if self.base_delay < 0 or self.max_delay < 0:
            raise ValueError("退避时间不能为负数")
        if not 0 <= self.jitter_ratio <= 1:
            raise ValueError("jitter_ratio 必须位于 0 到 1")
```

```python
class RetryPolicy:
    # dataclass 生成的 __init__ 完成字段赋值后会自动调用 __post_init__。
    # frozen 只限制赋值，不会自动校验数值范围，所以不变量仍需在这里显式检查。
    def __post_init__(self) -> None:
        if self.max_attempts < 1:
            raise ValueError("max_attempts 必须至少为 1")
        if self.base_delay < 0 or self.max_delay < 0:
            raise ValueError("退避时间不能为负数")
        if not 0 <= self.jitter_ratio <= 1:
            raise ValueError("jitter_ratio 必须位于 0 到 1")


# 该实例不可变，因此可安全地作为默认策略共享；若 dataclass 可变，就应避免共享默认对象。
DEFAULT_RETRY_POLICY = RetryPolicy()


# 数据库线程返回与 cursor/connection 脱钩的普通值对象。事件循环拿到它后无需再访问 SQLite 资源。
@dataclass(frozen=True, slots=True)
class DatabaseDocument:
    id: int
    title: str
    score: float


def print_section(title: str) -> None:
    # f-string 中的表达式先求值：`'=' * 72` 生成分隔线，再与标题一起插入最终字符串。
    print(f"\n{'=' * 72}\n{title}\n{'=' * 72}")


# 协程不能直接调用阻塞式 Event.wait()，否则服务线程没发信号时，整个事件循环也无法调度。
async def wait_for_thread_event(event: threading.Event, description: str) -> None:
    """threading.Event.wait 是同步方法，因此在线程中等，并保留有限清理上限。"""
    # to_thread 的第一个参数是尚未调用的普通函数，后面的 1.0 才是传给 event.wait 的实参。
    # await 等待工作线程返回布尔值期间，事件循环仍能运行其他 Task。
    reached: bool = await asyncio.to_thread(event.wait, 1.0)
    if not reached:
        raise RuntimeError(f"等待超时：{description}")
```

```
153
154 # 协程不能直接调用阻塞式 Event.wait()，否则服务线程没发信号时，整个事件循环也无法调度。
155 async def wait_for_thread_event(event: threading.Event, description: str) -> None:
156     """threading.Event.wait 是同步方法，因此在线程中等，并保留有限清理上限。"""
157     # to_thread 的第一个参数是尚未调用的普通函数，后面的 1.0 才是传给 event.wait 的实参。
158     # await 等待工作线程返回布尔值期间，事件循环仍能运行其他 Task。
159     reached: bool = await asyncio.to_thread(event.wait, 1.0)
160     if not reached:
161         raise RuntimeError(f"等待超时：{description}")
162
163
164 # 该函数只处理已经拿到的 Response，内部没有网络等待点，所以是普通 def 而不是 async def。
165 # 返回路径严格经过 HTTP 状态、JSON 语法、Document 合同三关，便于定位失败属于哪一层。
166 def document_from_response(response: httpx.Response) -> Document:
167     """按 HTTP → JSON → 模型三层检查；这些步骤本身都不是协程。"""
168     # raise_for_status 只检查内存中的状态码；4xx/5xx 时抛 HTTPStatusError。
169     try:
170         response.raise_for_status()
171     except httpx.HTTPStatusError as error:
172         # `from error` 建立显式异常链：外层接口稳定，日志又不会丢失底层失败证据。
173         raise HttpStatusFailure(f"HTTP 状态错误：{response.status_code}") from error
174
175     # 普通 client.get 已把正文读入 Response；json() 在当前线程把 bytes 解码成 dict 等 Python 值。
176     try:
177         payload: Any = response.json()
178     except json.JSONDecodeError as error:
179         raise ResponseJsonFailure("响应正文不是合法 JSON") from error
180
181     # 类型提示不会自动校验来自网络的 payload；Pydantic 在此真正执行字段、严格类型与范围规则。
182     try:
183         return Document.model_validate(payload)
184     except ValidationError as error:
185         raise ResponseValidationFailure("JSON 不符合 Document 模型") from error
186
187
188 # client 和 path/query 都由调用者提供：函数借用连接池发一次 GET，但不拥有也不关闭 client。
189 # 答名中的 `*` 表示它后面的 path 只能按名字传入，例如 path="/model"，不能作为第四个位置值。
```

```
187
188 # client 和 path/query 都由调用者提供：函数借用连接池发一次 GET，但不拥有也不关闭 client。
189 # 签名中的 `*` 表示它后面的 path 只能按名字传入，例如 path="/model"，不能作为第四个位置值。
190 async def fetch_document(
191     client: httpx.AsyncClient,
192     query: str,
193     *,
194     path: str = "/document",
195 ) -> Document:
196     """AsyncClient 由调用者拥有；工具函数只使用，不擅自关闭。"""
197     # 类型提示 `query: str` 主要服务类型检查器，运行时仍需手动拒绝非字符串与纯空白字符串。
198     # `or` 有短路行为：query 不是 str 时不会继续调用不存在的 strip()。
199     if not isinstance(query, str) or not query.strip():
200         raise ValueError("query 必须是非空字符串")
201     try:
202         # client.get 返回协程；只有 await 才真正等待连接、发送请求并读取非流式响应。
203         # 相对 path 会与 client 的 base_url 合并，params 字典编码为 `?query=...`。
204         response: Response = await client.get(path, params={"query": query})
205     except httpx.TransportError as error:
206         # 这里只转换“尚未获得有效 HTTP 响应”的传输层异常；状态/JSON/模型错误留给下一函数分层。
207         raise TransportFailure(f"HTTP 传输失败：{type(error).__name__}") from error
208     return document_from_response(response)
209
210
211 # `str | None` 表示请求头可能缺失；返回值也可能是秒数或 None。
212 # now 是仅限关键字的可选参数，测试可传固定时刻，生产调用默认读取当前 UTC 时间。
213 def retry_after_seconds(value: str | None, *, now: datetime | None = None) -> float | None:
214     """解析 Retry-After 的 delay-seconds 或 HTTP-date 两种标准形态。"""
215     if value is None:
216         return None
217     # 先去掉头值两端空白。纯十进制数字走 delay-seconds 快速路径，例如 "2" -> 2.0。
218     value = value.strip()
219     if value.isdigit():
220         return float(value)
221     try:
222         # 非纯数字时按 HTTP-date 解析；合法结果是 datetime，错误输入则返回 None 表示忽略该提示。
223         retry_at: datetime = parsedate_to_datetime(value)
```

```
213  def retry_after_seconds(value: str | None, *, now: datetime | None = None) -> float | None:
221      try:
222          # 非纯数字时按 HTTP-date 解析；合法结果是 datetime，错误输入则返回 None 表示忽略该提示。
223          retry_at: datetime = parsedate_to_datetime(value)
224      # Python 3.14 允许多个异常类直接以逗号分隔，不再强制写成 `(TypeError, ValueError, ...)`；
225      # 语义仍与 Java 的 multi-catch 相近，`as error` 只有需要读取异常对象时才必需。
226      except TypeError, ValueError, OverflowError:
227          return None
228      # 少数旧日期可能没有时区信息；这里只补成 UTC，确保后续能和带时区的 current 相减。
229      if retry_at.tzinfo is None:
230          retry_at: datetime = retry_at.replace(tzinfo=UTC)
231      current: datetime = now or datetime.now(UTC)
232      # 过去日期不能形成负 sleep，max 把它收敛成“无需额外等待”的 0 秒。
233      return max(0.0, (retry_at - current).total_seconds())
234
235
236  # 根据“第几次失败”计算下一轮等待。response 为 None 表示传输阶段失败、根本没有响应头可读。
237  def next_backoff(
238      policy: RetryPolicy,
239      failed_attempt: int,
240      response: httpx.Response | None,
241      randomizer: random.Random,
242  ) -> float:
243      """指数退避加正向 jitter，同时考虑服务端 Retry-After 提示。"""
244      # 第 1、2、3 次失败对应 base_delay * 1、2、4；`**` 是乘方运算符。
245      exponential: Any = policy.base_delay * (2 ** (failed_attempt - 1))
246      # random() 返回 [0.0, 1.0)；这里只加正向随机抖动，避免大量客户端同时重试。
247      jitter: Any = exponential * policy.jitter_ratio * randomizer.random()
248      client_delay: Any = exponential + jitter
249      # 条件表达式只在确实拿到 Response 时读取 Retry-After，否则 server_delay 为 None。
250      server_delay: float | None = retry_after_seconds(response.headers.get("Retry-After")) if response else None
251      # `server_delay or 0.0` 把 None 和合法的 0 秒都转为 0；先尊重更长的一方，再受本地上限约束。
252      requested_delay: Any = max(client_delay, server_delay or 0.0)
253      return min(requested_delay, policy.max_delay)
254
255
```

```
254
255
256 # 这是本章的核心重试状态机。client、limiter 和绝对 deadline 都由更外层所有者注入，
257 # 可选的 policy/randomizer 控制策略与可重复性，trace/Event 只暴露观察点而不改变成功结果。
258 # `*` 后的所有参数都必须写出名字，调用点不易把多个 float/策略对象按位置传错。
259 async def fetch_read_only_with_retry(
260     client: httpx.AsyncClient,
261     query: str,
262     *,
263     limiter: asyncio.Semaphore,
264     deadline: float,
265     policy: RetryPolicy = DEFAULT_RETRY_POLICY,
266     randomizer: random.Random | None = None,
267     trace: list[str] | None = None,
268     backoff_started: asyncio.Event | None = None,
269 ) -> Document:
270     """在共同 deadline 内有限重试一个按合同只读的 GET。
271
272     每次网络尝试才持有 Semaphore；退避前已经退出 async with，避免所有许可都被
273     睡眠中的重试占住。模型/参数错误不会进入重试分支。
274     """
275     # 重试不会让永久性的参数错误自行恢复，因此在进入循环、消耗并发许可前先拒绝它。
276     if not isinstance(query, str) or not query.strip():
277         raise ValueError("query 必须是非空字符串，参数错误不会重试")
278
279     # loop.time() 是单调时钟，不受系统墙上时间校准影响；传入的 deadline 必须来自同一时钟。
280     loop: AbstractEventLoop = asyncio.get_running_loop()
281     # 调用者可注入固定随机源让实验可重复；未传时才创建独立 Random 实例。
282     rng: Random = randomizer or random.Random()
283     # None 表示调用者不关心轨迹，此时创建新列表。不能写 `trace or []`，因为传入的空列表
284     # 本来就是要由函数原地追加并让调用者观察的对象，按真值选择会错误地换掉它。
285     events: list[str] = trace if trace is not None else []
286     last_failure = "尚未尝试"
287
288     # range 的停止值不包含在结果内，所以这里生成 1..max_attempts，attempt 可直接用于日志。
289     for attempt in range(1, policy.max_attempts + 1):
290         # deadline 是一次逻辑调用共享的绝对时刻；每轮只重算剩余量，不重新发一份完整预算。
```

```python
async def fetch_read_only_with_retry(
    # range 的停止值不包含在结果内，所以这里生成 1..max_attempts，attempt 可直接用于日志。
    for attempt in range(1, policy.max_attempts + 1):
        # deadline 是一次逻辑调用共享的绝对时刻；每轮只重算剩余量，不重新发一份完整预算。
        remaining: float = deadline - loop.time()
        if remaining <= 0:
            raise RetryBudgetExceeded("发送下一次尝试前，整体预算已耗尽")

        # 每轮先假定还没拿到响应。若 client.get 抛传输错误，后续退避不能读取响应头。
        response: httpx.Response | None = None
        retryable_failure = False
        try:
            # deadline 是所有尝试和退避共享的绝对时刻，不会在每轮重新发满预算。
            # timeout_at 超时会先取消当前 Task；离开上下文时将这次内部取消转换成 TimeoutError。
            # limiter 只包一次在途传输，退出 async with 时即使异常/取消也会自动归还许可。
            async with asyncio.timeout_at(deadline):
                async with limiter:
                    events.append(f"尝试 {attempt}: 获取许可，剩余预算 {remaining:.3f}s")
                    response = await client.get("/document", params={"query": query})
        except TimeoutError as error:
            # 这里只捕获本层 timeout_at 转出的 TimeoutError；外部 Task.cancel() 产生的
            # CancelledError 不属于该分支，会原样向上传播。
            raise RetryBudgetExceeded("一次传输等待越过了整体截止时间") from error
        except RETRYABLE_TRANSPORT_ERRORS as error:
            # 白名单中的异常说明没有可用响应，但只要次数和预算允许，仍可退避后再尝试。
            retryable_failure = True
            last_failure: str = type(error).__name__
            events.append(f"尝试 {attempt}: 可重试传输失败 {last_failure}")
        else:
            # `else` 只在 try 没抛异常时执行，此时 response 已由 client.get 绑定。
            if response.status_code in RETRYABLE_STATUS_CODES:
                retryable_failure = True
                last_failure: str = f"HTTP {response.status_code}"
                events.append(f"尝试 {attempt}: 可重试状态 {response.status_code}")
            else:
                # 400、JSON 错误、模型错误都在这里立即抛出，不会被重试循环吞掉。
```

```
259  async def fetch_read_only_with_retry(
321              else:
322                  # 400、JSON 错误、模型错误都在这里立即抛出，不会被重试循环吞掉。
323                  # document_from_response 若返回，得到的已是校验后的 Document，可立即结束整个函数。
324                  return document_from_response(response)
325
326          # 当前实现所有正常落到此处的路径都应已标记"可重试"；断言用于尽早暴露状态机维护错误。
327          if not retryable_failure:
328              raise AssertionError("重试状态机进入了不可能的分支")
329          # 最后一次失败后不再计算/等待退避，直接把最后失败类别写入稳定异常。
330          if attempt == policy.max_attempts:
331              raise RetryExhausted(f"{policy.max_attempts} 次尝试后仍失败；最后失败={last_failure}")
332
333          # 把这次 Response 交给退避函数后，429/503 的 Retry-After 才可能影响等待时间。
334          delay: float = next_backoff(policy, attempt, response, rng)
335          remaining: float = deadline - loop.time()
336          # `>=` 也拒绝：若 sleep 恰好吃光全部剩余时间，已经没有预算发送下一次请求。
337          if delay >= remaining:
338              raise RetryBudgetExceeded(
339                  f"剩余预算 {max(remaining, 0.0):.3f}s 不足以完成 {delay:.3f}s 退避"
340              )
341
342          # 此处已经离开 limiter 上下文。sleep 本身是可取消的等待点。
343          events.append(f"尝试 {attempt}: 已释放许可，退避 {delay:.3f}s")
344          # 可选 Event 只用于测试精确知道任务已进入退避，不靠固定 sleep 猜调度时机。
345          if backoff_started is not None:
346              backoff_started.set()
347          try:
348              # 退避也受同一个 deadline 约束。await sleep 会把控制权交还事件循环，且可被取消。
349              async with asyncio.timeout_at(deadline):
350                  await asyncio.sleep(delay)
351          except TimeoutError as error:
352              raise RetryBudgetExceeded("退避期间整体预算耗尽") from error
353
354      # 上面的 return/raise 覆盖有限循环的所有出口；保留断言可防止以后改状态机时静默返回 None。
355      raise AssertionError("有限 for 循环不应从末尾落出")
```

```
357
358 # 第一节使用真实回环服务验证客户端生命周期、连接复用、两类 timeout 和流关闭，
359 # 再用进程内 MockTransport 精确制造四层错误。fixture 由更外层 run_examples 创建并拥有。
360 async def demonstrate_httpx(fixture: LocalHttpFixture) -> None:
361     print_section("第一节：AsyncClient、连接池、分层错误与 JSONL 流")
362     # state 是共享状态对象的另一个名字，并没有复制；客户端协程和 fixture 服务线程用其中的
363     # Event 协调先后，用请求记录观察服务端真正收到的连接。
364     state: FixtureState = fixture.state
365
366     # AsyncClient 是异步上下文管理器：进入后可复用内部连接池，离开时自动 await 关闭资源。
367     # base_url 让下方只写相对路径；trust_env=False 避免本机代理环境变量干扰回环实验。
368     async with httpx.AsyncClient(
369         base_url=fixture.base_url,
370         timeout=HTTP_TIMEOUT,
371         limits=HTTP_LIMITS,
372         trust_env=False,
373     ) as client:
374         # fixture 可能已经服务过其他小节。先记当前长度，之后切片只分析本节新增请求。
375         record_start: int = len(state.records_snapshot())
376
377         # 第一条请求拆开写，便于后面展示 Response 的同步解析；第二条走可复用工具函数。
378         response: Response = await client.get("/document", params={"query": "HTTPX"})
379         first: Document = document_from_response(response)
380         second: Document = await fetch_document(client, "MCP")
381
382         # inspect 检查函数实现是否声明为协程函数：client.get 需要 await，response.json 不需要。
383         # 最后一行读取的是两个通过 Pydantic 校验的 Document.title 字段。
384         print("1) client.get 是协程方法:", inspect.iscoroutinefunction(client.get))
385         print("   response.json 是普通方法:", not inspect.iscoroutinefunction(response.json))
386         print("   两次模型结果:", first.title, "|", second.title)
387
388         # 切片 `[record_start:]` 取起点到末尾的新列表；列表推导式再只保留 /document 请求。
389         # 推导式可类比一段紧凑的 Java Stream filter，但过长时普通 for 往往更清楚。
390         document_records: list[RequestRecord] = [
391             record
392             for record in state.records_snapshot()[record_start:]
393             if record.path == "/document"
```

```python
async def demonstrate_httpx(fixture: LocalHttpFixture) -> None:

        # 切片 `[record_start:]` 取起点到末尾的新列表；列表推导式再只保留 /document 请求。
        # 推导式可类比一段紧凑的 Java Stream filter，但过长时普通 for 往往更清楚。
        document_records: list[RequestRecord] = [
            record
            for record in state.records_snapshot()[record_start:]
            if record.path == "/document"
        ]
        # 每条服务端记录保存客户端源端口；两次端口相同是本次受控实验复用 TCP 连接的证据。
        ports: list[int] = [record.client_port for record in document_records]
        print("   服务端观察到的客户端端口:", ports)
        print("   本次本地实验复用了连接:", len(ports) == 2 and ports[0] == ports[1])
        # assert 记录教学实验必须满足的不变量；若连接没有按预期复用，立即停止而非只打印 False。
        assert len(ports) == 2 and ports[0] == ports[1]

        # 打印配置是为了把阶段 timeout 与池容量放在一起观察；它们都不是整个操作的 deadline。
        print("2) timeout:", HTTP_TIMEOUT)
        print("   limits: max_connections=1, max_keepalive_connections=1")

        # create_task 立即把慢请求注册进事件循环并返回 Task；此处先不 await 它，
        # 才能同时等待服务端 Event，确定请求已到达后再观察 read timeout。
        slow_task: Task[Response] = asyncio.create_task(client.get("/slow"), name="read-timeout-demo")
        await wait_for_thread_event(state.slow_request_started, "服务端收到慢请求")
        try:
            await slow_task
        except httpx.ReadTimeout as error:
            # ReadTimeout 表示在 read 阶段等待下一块数据过久，不代表总 deadline 或连接池等待。
            print("   慢响应结果:", type(error).__name__, "(read 阶段无新数据)")
        else:
            # try 没抛异常才走 else；慢端点若意外成功，实验条件已失效，因此主动报错。
            raise AssertionError("慢响应应触发 ReadTimeout")
        finally:
            # finally 无论成功、预期异常还是断言都执行，放行服务线程，避免它永久卡在 Event.wait。
            state.release_slow_response.set()
        # 客户端超时只结束客户端等待；服务线程是独立执行单元，还需确认其清理完成。
        await wait_for_thread_event(state.slow_request_finished, "慢请求服务线程清理")
```

```python
def validate_httpx(fixture: LocalHttpFixture) -> None:

    # 上一个慢请求已结束后才能重置三个 Event，给池耗尽实验建立全新的同步点。
    state.reset_slow_exchange()
    # 占池请求把 read timeout 放宽到 1 秒，保证它在第二个请求等待连接池时仍持有唯一连接。
    holder_timeout = httpx.Timeout(connect=0.2, read=1.0, write=0.2, pool=0.03)
    holder: Task[Response] = asyncio.create_task(
        client.get("/slow", timeout=holder_timeout),
        name="pool-holder",
    )
    await wait_for_thread_event(state.slow_request_started, "首个请求占用唯一连接")
    try:
        # HTTP_LIMITS 只有一条连接且已被 holder 占用；第二个请求最多等待 pool=0.03 秒。
        await client.get("/document", params={"query": "等待连接池"})
    except httpx.PoolTimeout as error:
        print("    连接池耗尽结果:", type(error).__name__, " (等待池中连接超时) ")
    else:
        raise AssertionError("唯一连接被占用时，第二个请求应触发 PoolTimeout")
    finally:
        state.release_slow_response.set()
    # 放行后 holder 应正常取得 200 响应。Task 保存协程的最终响应，await 才读取该结果。
    holder_response: Response = await holder
    assert holder_response.status_code == HTTPStatus.OK
    await wait_for_thread_event(state.slow_request_finished, "占池请求清理")

    # 先在外层保留 Response 引用，才能对比 stream 上下文内外的 is_closed 状态。
    # `T | None` 说明进入上下文前还没有响应对象，也是静态检查器要求下方先判空的原因。
    streamed_response: httpx.Response | None = None
    # client.stream 不预先读完整正文；上下文拥有这次 Response，并在退出时异步关闭它。
    async with client.stream("GET", "/events") as stream_response:
        streamed_response = stream_response
        stream_response.raise_for_status()
        # aiter_lines() 以协议行作为消费边界。`async for` 每次可能等待网络数据，
        # 但仍是逐行串行消费，不会因为写了 async 自动并行处理多行。
        async for line in stream_response.aiter_lines():
            # line 已是不含换行符的 str；json.loads 每次把一条完整 JSONL 记录解析成 dict。
            event: Any = json.loads(line)
```

```
360  async def demonstrate_httpx(fixture: LocalHttpFixture) -> None:
454              # aiter_lines() 以协议行作为消费边界。`async for` 每次可能等待网络数据，
455              # 但仍是逐行串行消费，不会因为写了 async 自动并行处理多行。
456              async for line in stream_response.aiter_lines():
457                  # line 已是不含换行符的 str；json.loads 每次把一条完整 JSONL 记录解析成 dict。
458                  event: Any = json.loads(line)
459                  print("3) 收到一条完整 JSONL 事件:", event)
460                  # 刻意只消费第一行，验证提前停止时外层 stream 上下文仍负责关闭响应。
461                  break
462              assert state.first_stream_line_sent.is_set()
463              print("   stream 上下文内 response.is_closed:", stream_response.is_closed)
464          # 离开 async with 后 Response 应已关闭。`and` 会短路，None 时不会访问 `.is_closed`。
465          assert streamed_response is not None and streamed_response.is_closed
466          print("   提前退出后 response.is_closed:", streamed_response.is_closed)
467          # 客户端已经关闭响应流，现在才允许服务端尝试发送尾部，并等待其处理预期断连后退出。
468          state.release_stream_tail.set()
469          await wait_for_thread_event(state.stream_request_finished, "流服务线程清理")
470
471      # 这里已经离开 AsyncClient 的 async with，但变量仍可引用对象；is_closed 用于验证资源状态。
472      print("4) 客户端作用域退出后 client.is_closed:", client.is_closed)
473
474      # MockTransport handler 是普通同步函数：每次请求按 path 返回指定 Response 或抛指定异常。
475      # 它不进行 DNS/TCP，也不能用来证明连接池和真实网络 timeout 行为。
476      def layered_handler(request: httpx.Request) -> httpx.Response:
477          # match/case 是结构化模式匹配；这里对字符串值分支，作用近似 Java 的 switch expression。
478          match request.url.path:
479              case "/transport":
480                  raise httpx.ConnectError("受控连接失败", request=request)
481              case "/status":
482                  return httpx.Response(404, json={"detail": "missing"})
483              case "/json":
484                  return httpx.Response(200, content=b"not-json")
485              case "/model":
486                  return httpx.Response(
487                      200,
488                      json={"id": "25", "title": "错误类型", "score": 0.95},
489                  )
```

```python
async def demonstrate_httpx(fixture: LocalHttpFixture) -> None:
    def layered_handler(request: httpx.Request) -> httpx.Response:
            case "/model":
                return httpx.Response(
                    200,
                    json={"id": "25", "title": "错误类型", "score": 0.95},
                )
            case _:
                # `_` 是兜底通配模式，任何未列出的路径都会来到这里。
                raise AssertionError(f"未预期的 mock 路径：{request.url.path}")

    print("5) MockTransport 分层错误（不经过 DNS/TCP/连接池）:")
    # transport 注入进程内 handler；base_url 仍是必需的，因为 fetch_document 发送相对路径。
    async with httpx.AsyncClient(
        base_url="https://mock.invalid",
        transport=httpx.MockTransport(layered_handler),
    ) as mock_client:
        # 每个元素都是 `(请求路径，预期应用异常类)` 二元组。
        # Sequence[...] 是只读使用方式的类型合同，内层 `type[Exception]` 表示保存的是异常类本身，
        # 不是某个异常实例；外层实际值选 tuple，使测试用例在运行中不会被追加或删除。
        cases: Sequence[tuple[str, type[Exception]]] = (
            ("/transport", TransportFailure),
            ("/status", HttpStatusFailure),
            ("/json", ResponseJsonFailure),
            ("/model", ResponseValidationFailure),
        )
        # for 将每个二元组按位置解包为 path 和 expected_type，无需手写 case[0]/case[1]。
        for path, expected_type in cases:
            try:
                await fetch_document(mock_client, "demo", path=path)
            # except 后也可以放运行时变量，只要它的值是异常类或异常类元组。
            except expected_type as error:
                # 前面用 raise ... from ... 保留了底层异常；此处读取 __cause__ 验证异常分层未丢证据。
                cause_name: str = type(error.__cause__).__name__
                print(f"   {path:<10} -> {type(error).__name__}, cause={cause_name}")
            else:
```

```
360  async def demonstrate_httpx(fixture: LocalHttpFixture) -> None:
518              else:
519                  raise AssertionError(f"{path} 应抛 {expected_type.__name__}")
520
521
522  # 第二节全部使用 MockTransport，让“第几次发生哪种失败”完全确定；它验证的是重试状态机，
523  #、不是实际网络阶段。五组场景依次覆盖：暂时失败后成功、次数耗尽、预算不足、退避中取消，
524  # 以及不可重试状态立即失败。
525  async def demonstrate_retries() -> None:
526      print_section("第二节：有限重试、共同预算与取消")
527
528      # 整数不可变；内层 handler 若要重新绑定外层函数的局部变量，必须用 nonlocal 声明。
529      transient_attempts = 0
530
531      def transient_handler(request: httpx.Request) -> httpx.Response:
532          nonlocal transient_attempts
533          transient_attempts += 1
534          # 同一个 handler 按调用次数制造“连接失败 -> 503 -> 200”，使成功必然发生在第三次。
535          if transient_attempts == 1:
536              raise httpx.ConnectError("第一次连接失败", request=request)
537          if transient_attempts == 2:
538              return httpx.Response(503, headers={"Retry-After": "0"})
539          return httpx.Response(
540              200,
541              json={"id": 25, "title": "第三次成功", "score": 0.99},
542          )
543
544      # trace 是可变列表，会被重试函数原地追加；调用结束后本函数仍可看到同一个列表中的事件。
545      trace: list[str] = []
546      # 容量 1 代表同一时刻最多一次网络尝试；每次退避时许可会释放。
547      limiter = asyncio.Semaphore(1)
548      async with httpx.AsyncClient(
549          base_url="https://mock.invalid",
550          transport=httpx.MockTransport(transient_handler),
551      ) as client:
552          # deadline 只在逻辑调用入口计算一次。固定种子 Random 让 jitter 序列可重复，
553          # 但示例只依赖次数和上限，不把某个随机小数当作业务合同。
```

```
525 async def demonstrate_retries() -> None:
544     # trace 是可变列表，会被重试函数原地追加；调用结束后本函数仍可看到同一个列表中的事件。
545     trace: list[str] = []
546     # 容量 1 代表同一时刻最多一次网络尝试；每次退避时许可会释放。
547     limiter = asyncio.Semaphore(1)
548     async with httpx.AsyncClient(
549         base_url="https://mock.invalid",
550         transport=httpx.MockTransport(transient_handler),
551     ) as client:
552         # deadline 只在逻辑调用入口计算一次。固定种子 Random 让 jitter 序列可重复，
553         # 但示例只依赖次数和上限，不把某个随机小数当作业务合同。
554         document: Document = await fetch_read_only_with_retry(
555             client,
556             "read-only",
557             limiter=limiter,
558             deadline=asyncio.get_running_loop().time() + 1.0,
559             policy=RetryPolicy(base_delay=0.001, max_delay=0.005, jitter_ratio=0.25),
560             randomizer=random.Random(25),
561             trace=trace,
562         )
563     # client 已关闭，Document 和 trace 是与连接资源脱钩的普通 Python 对象，仍可安全读取。
564     print("1) 暂时失败后成功:", document.title, f"attempts={transient_attempts}")
565     # 每轮 event 是一条 str；print 的多个参数默认用空格分隔。
566     for event in trace:
567         print("  ", event)
568     assert transient_attempts == 3
569
570     # 第二个 handler 永远返回白名单状态 503，用计数器验证 max_attempts 指总尝试数。
571     always_attempts = 0
572
573     def always_fails(request: httpx.Request) -> httpx.Response:
574         nonlocal always_attempts
575         always_attempts += 1
576         return httpx.Response(503, headers={"Retry-After": "0"}, request=request)
577
578     async with httpx.AsyncClient(
```

```
525  async def demonstrate_retries() -> None:
577
578      async with httpx.AsyncClient(
579          base_url="https://mock.invalid",
580          transport=httpx.MockTransport(always_fails),
581      ) as client:
582          try:
583              # 退避参数全为 0，让实验无需真实等待；有限次数规则仍照常执行。
584              await fetch_read_only_with_retry(
585                  client,
586                  "always-fail",
587                  limiter=asyncio.Semaphore(1),
588                  deadline=asyncio.get_running_loop().time() + 1.0,
589                  policy=RetryPolicy(base_delay=0, max_delay=0, jitter_ratio=0),
590              )
591          except RetryExhausted as error:
592              # f-string 只负责把实际计数嵌入输出；下一条 assert 才正式验证次数。
593              print("2) 一直失败:", error, f"actual_attempts={always_attempts}")
594          else:
595              raise AssertionError("一直返回 503 应耗尽重试")
596      assert always_attempts == 3
597
598      # 第三个 handler 要求等待 2 秒，但调用总预算只有约 0.05 秒。
599      # 预期在第一次响应后立刻拒绝退避，不发送第二次请求。
600      budget_attempts = 0
601
602      def asks_for_long_wait(request: httpx.Request) -> httpx.Response:
603          nonlocal budget_attempts
604          budget_attempts += 1
605          return httpx.Response(503, headers={"Retry-After": "2"}, request=request)
606
607      async with httpx.AsyncClient(
608          base_url="https://mock.invalid",
609          transport=httpx.MockTransport(asks_for_long_wait),
610      ) as client:
611          try:
612              await fetch_read_only_with_retry(
```

```python
async def demonstrate_retries() -> None:
    async with httpx.AsyncClient(
        base_url="https://mock.invalid",
        transport=httpx.MockTransport(asks_for_long_wait),
    ) as client:
        try:
            await fetch_read_only_with_retry(
                client,
                "budget",
                limiter=asyncio.Semaphore(1),
                deadline=asyncio.get_running_loop().time() + 0.05,
                policy=RetryPolicy(base_delay=0.01, max_delay=2.0, jitter_ratio=0),
            )
        except RetryBudgetExceeded as error:
            print("3) 剩余预算不足:", error, f"attempts={budget_attempts}")
        else:
            raise AssertionError("Retry-After 大于剩余预算时不应再睡眠重试")
    assert budget_attempts == 1

    # 第四个 handler 要求长退避，给外层留下稳定窗口：先确认许可已释放，再主动取消 Task。
    cancelled_attempts = 0

    def waits_in_backoff(request: httpx.Request) -> httpx.Response:
        nonlocal cancelled_attempts
        cancelled_attempts += 1
        return httpx.Response(503, headers={"Retry-After": "10"}, request=request)

    # asyncio.Event 只在同一事件循环内协调协程；这里无需跨线程，所以不用 threading.Event。
    backoff_started = asyncio.Event()
    cancellation_limiter = asyncio.Semaphore(1)
    async with httpx.AsyncClient(
        base_url="https://mock.invalid",
        transport=httpx.MockTransport(waits_in_backoff),
    ) as client:
        # create_task 让重试协程与当前演示协程并发推进；name 只用于调试和任务观测。
        retry_task: Task[Document] = asyncio.create_task(
            fetch_read_only_with_retry(
```

```python
async def demonstrate_retries() -> None:
    async with httpx.AsyncClient(
        base_url="https://mock.invalid",
        transport=httpx.MockTransport(waits_in_backoff),
    ) as client:
        # create_task 让重试协程与当前演示协程并发推进；name 只用于调试和任务观测。
        retry_task: Task[Document] = asyncio.create_task(
            fetch_read_only_with_retry(
                client,
                "cancel-me",
                limiter=cancellation_limiter,
                deadline=asyncio.get_running_loop().time() + 20.0,
                policy=RetryPolicy(base_delay=1, max_delay=10, jitter_ratio=0),
                backoff_started=backoff_started,
            ),
            name="cancel-during-backoff",
        )
        # handler 返回 503 后，重试函数释放 Semaphore、设置 Event 并进入 sleep，才会唤醒这里。
        await backoff_started.wait()

        # 若退避仍占着唯一许可，这里会超时。成功获取是释放时机的运行证据。
        async with asyncio.timeout(0.2):
            await cancellation_limiter.acquire()
        # 手动 acquire 必须配对 release；Semaphore 不是线程锁，没有 with 块替我们自动释放这一次。
        cancellation_limiter.release()
        permit_was_free = True

        # cancel() 只是提出取消请求；必须继续 await Task，才能观察它最终传播的 CancelledError。
        retry_task.cancel()
        try:
            await retry_task
        except asyncio.CancelledError:
            # 重试函数没有把外部取消改写成预算或重试失败；这也是协作取消必须保留的语义。
            print(
                "4) 退避中整体取消: CancelledError,",
                f"attempts={cancelled_attempts}, 退避未占许可={permit_was_free}",
            )
```

```python
async def demonstrate_retries() -> None:
            )
        else:
            raise AssertionError("取消退避任务后应继续传播 CancelledError")

    # 最后一组用 400 证明不可重试 HTTP 状态会直接经过分层校验，只发一次请求。
    non_retry_attempts = 0

    def bad_request(request: httpx.Request) -> httpx.Response:
        nonlocal non_retry_attempts
        non_retry_attempts += 1
        return httpx.Response(400, json={"detail": "bad query"}, request=request)

    async with httpx.AsyncClient(
        base_url="https://mock.invalid",
        transport=httpx.MockTransport(bad_request),
    ) as client:
        try:
            await fetch_read_only_with_retry(
                client,
                "invalid",
                limiter=asyncio.Semaphore(1),
                deadline=asyncio.get_running_loop().time() + 1.0,
            )
        except HttpStatusFailure as error:
            print("5) 参数/HTTP 400 不重试:", error, f"attempts={non_retry_attempts}")
        else:
            raise AssertionError("HTTP 400 应立即失败")
    assert non_retry_attempts == 1


# 这是一个极小协程：Task 一旦得到事件循环执行机会，就设置跨线程可见的 heartbeat。
# 它没有 await，因此开始运行后会一直执行到函数返回。
async def set_heartbeat(state: FixtureState) -> None:
    state.event_loop_heartbeat.set()
```

```python
# create_database 是交给 to_thread 的同步工作单元：连接在调用它的工作线程中创建、使用并关闭。
def create_database(database_path: Path) -> None:
    """初始化也在工作线程完成；连接不会泄漏到事件循环线程。"""
    # sqlite3.Connection 自身的 with 只管理事务，不负责 close；closing 明确关闭连接。
    # 一个 with 可同时进入多个上下文：先由 closing 保证最终 close，再由 connection 管理事务。
    # 正常退出时事务提交；异常退出时回滚；最后无论哪条路径都会关闭连接。
    with closing(sqlite3.connect(database_path, timeout=0.5)) as connection, connection:
        # execute 创建表；数据库路径来自 TemporaryDirectory，不会覆盖固定业务数据库。
        connection.execute(
            "CREATE TABLE documents (id INTEGER PRIMARY KEY, title TEXT, score REAL)"
        )
        # executemany 对三个参数元组重复执行同一条参数化 SQL。`?` 由 sqlite3 绑定，
        # 不要用 f-string 拼接外部值，否则会混淆 SQL 与数据并引入注入风险。
        connection.executemany(
            "INSERT INTO documents VALUES (?, ?, ?)",
            (
                (1, "asyncio 取消", 0.92),
                (2, "HTTPX 连接池", 0.97),
                (3, "普通同步代码", 0.61),
            ),
        )


# 查询也封装成完整同步工作单元。返回值是新建 dataclass 列表，不携带连接或 cursor。
def query_database(database_path: Path, minimum_score: float) -> list[DatabaseDocument]:
    """完整查询在线程内创建、使用并关闭同一条 sqlite3 连接。"""
    with closing(sqlite3.connect(database_path, timeout=0.5)) as connection:
        # `(minimum_score,)` 尾部逗号不可省：它创建单元素 tuple，供 SQL 中唯一的 `?` 绑定。
        # fetchall 在关闭连接前把全部结果物化为行列表，不把惰性 cursor 带回事件循环线程。
        rows: list[Any] = connection.execute(
            "SELECT id, title, score FROM documents WHERE score >= ? ORDER BY score DESC",
            (minimum_score,),
        ).fetchall()
    # 列表推导式逐行构造与数据库资源脱钩的值对象，并显式归一化 SQLite 返回的基础类型。
    return [DatabaseDocument(int(row[0]), str(row[1]), float(row[2])) for row in rows]
```

```python
# 这个普通函数模拟阻塞式 SDK：started/finished 暴露线程生命周期，may_finish 是合作式放行闸门，
# side_effects 则证明即使等待它的 asyncio Task 已取消，底层线程仍可能产生可见副作用。
def lingering_sync_work(
    started: threading.Event,
    may_finish: threading.Event,
    finished: threading.Event,
    side_effects: list[str],
) -> None:
    """模拟无法由 Task.cancel() 强停的同步 SDK 调用。"""
    # threading.Event.set() 可从工作线程唤醒其他线程中的 wait()。
    started.set()
    try:
        # Event.wait 是同步阻塞，最多等 2 秒作为清理保险；返回 True 表示确实被 may_finish 放行。
        if may_finish.wait(timeout=2.0):
            side_effects.append("线程中的底层工作仍然完成")
    finally:
        # 不论正常放行还是函数异常，所有者都能通过 finished 判断线程函数已经离开。
        finished.set()


# 第三节先故意在 async 函数中直调 Requests，随后用 to_thread 正确适配；
# 再把同一边界应用到 SQLite，并观察“取消 asyncio Task 不会强停线程”。
async def demonstrate_sync_io(fixture: LocalHttpFixture) -> None:
    print_section("第三节：同步库、sqlite3 与 asyncio.to_thread")
    state: FixtureState = fixture.state
    # f-string 把 fixture 动态选择的本机端口插入 URL；结果是普通 str。
    url: str = f"{fixture.base_url}/sync-wait"

    # clear 建立“尚未心跳”的初始状态；create_task 只是把协程排入事件循环，
    # 只有当前协程让出控制权后，heartbeat Task 才真正有机会运行。
    state.event_loop_heartbeat.clear()
    direct_heartbeat: Task[None] = asyncio.create_task(set_heartbeat(state))
    # 教学反例：async 函数里直接调用同步 requests，当前事件循环线程一直被占用。
    # 此处没有 await；requests.get 从调用到返回独占当前线程。元组 timeout 分别表示连接和读取等待。
    direct_response: Response = requests.get(url, timeout=(0.2, 1.0))  # noqa: ASYNC210
    # Requests 已缓冲响应正文，json() 是当前线程中的普通解析；close 明确归还其底层连接资源。
```



python-quickstart-for-java

example.py .../14_data_models × example.py .../15_protocols_generics × example.py .../16_decorators × example.py .../25_async_http_lo ×

java_to_python > chapters > 25_async_http_io > example.py > next_backoff

```
766  async def demonstrate_sync_io(fixture: LocalHttpFixture) -> None:
779      # Requests 已缓冲响应正文，json() 是当前线程中的普通解析；close 明确归还其底层连接资源。
780      direct_payload: Any = direct_response.json()
781      direct_response.close()
782      # 直到 requests.get 返回后，事件循环才终于能运行刚才计划的 heartbeat Task。
783      await direct_heartbeat
784      print("1) async 内直接 requests，服务端及时看到 heartbeat:", direct_payload["heartbeat_seen"])
785      # 服务端只等 0.25 秒；期间事件循环被同步请求堵住，所以观察值稳定为 False。
786      assert direct_payload == {"heartbeat_seen": False}
787
788      # 重置相同实验。这次工作线程负责阻塞式 HTTP，事件循环可调度 heartbeat Task。
789      state.event_loop_heartbeat.clear()
790      threaded_heartbeat: Task[None] = asyncio.create_task(set_heartbeat(state))
791      # to_thread 接受普通同步函数以及它的位置/关键字参数；不要把调用结果传进去。
792      # `requests.get` 在这里是函数对象，url 是位置参数，timeout 是关键字参数；调用发生在线程池中。
793      threaded_response: Response = await asyncio.to_thread(
794          requests.get,
795          url,
796          timeout=(0.2, 1.0),
797      )
798      # await to_thread 只把同步等待移出了事件循环；返回的仍是普通 Requests Response，照常解析和关闭。
799      threaded_payload: Any = threaded_response.json()
800      threaded_response.close()
801      await threaded_heartbeat
802      print(
803          "   requests 放入 to_thread，服务端及时看到 heartbeat:", threaded_payload["heartbeat_seen"]
804      )
805      assert threaded_payload == {"heartbeat_seen": True}
806
807      # TemporaryDirectory 是同步上下文管理器：进入时创建唯一临时目录，退出时递归清理其中的数据库。
808      with tempfile.TemporaryDirectory(prefix="chapter25-") as temp_directory:
809          # `/` 被 Path 重载为路径拼接，生成 Path 对象而不是做数值除法。
810          database_path: Path = Path(temp_directory) / "documents.sqlite3"
811          # 传入的是函数对象和参数；两次工作各在线程中打开并关闭自己的 sqlite3 连接。
812          await asyncio.to_thread(create_database, database_path)
813          rows: list[DatabaseDocument] = await asyncio.to_thread(query_database, database_path, 0.90)
814          print("2) sqlite3 线程内完整查询:", rows)
```

```python
async def demonstrate_sync_io(fixture: LocalHttpFixture) -> None:
    # TemporaryDirectory 是同步上下文管理器：进入时创建唯一临时目录，退出时递归清理其中的数据库。
    with tempfile.TemporaryDirectory(prefix="chapter25-") as temp_directory:
        # `/` 被 Path 重载为路径拼接，生成 Path 对象而不是做数值除法。
        database_path: Path = Path(temp_directory) / "documents.sqlite3"
        # 传入的是函数对象和参数；两次工作各在线程中打开并关闭自己的 sqlite3 连接。
        await asyncio.to_thread(create_database, database_path)
        rows: list[DatabaseDocument] = await asyncio.to_thread(query_database, database_path, 0.90)
        print("2) sqlite3 线程内完整查询:", rows)
        # 推导式取出排序后的 id，验证 SQL 的阈值过滤和 `ORDER BY score DESC`。
        assert [row.id for row in rows] == [2, 1]

    # 三个线程 Event 把“已开始、允许结束、确实结束”分开，避免依赖碰运气的 sleep。
    started = threading.Event()
    may_finish = threading.Event()
    finished = threading.Event()
    side_effects: list[str] = []
    # to_thread 返回 coroutine；再用 create_task 包装后，才能稍后单独对等待方调用 cancel()。
    thread_task: Task[None] = asyncio.create_task(
        asyncio.to_thread(lingering_sync_work, started, may_finish, finished, side_effects),
        name="to-thread-cancellation-demo",
    )
    # 先确认工作线程已进入函数，避免在尚未提交/启动前就取消而得出错误结论。
    await wait_for_thread_event(started, "同步线程开始工作")
    # 取消的是 asyncio Task 的等待关系，不会向正在运行的普通 Python 线程注入异常。
    thread_task.cancel()
    try:
        await thread_task
    except asyncio.CancelledError:
        print("3) 等待 to_thread 的 Task 已取消；工作线程已结束:", finished.is_set())
    else:
        raise AssertionError("等待 to_thread 的 Task 应进入取消状态")
    # Task 已传播取消时，底层函数仍堵在 may_finish.wait，因此 finished 必须尚未设置。
    assert not finished.is_set()

    # 当前示例仍拥有底层工作的释放事件，所以必须放行并确认线程结束。
```

```
766  async def demonstrate_sync_io(fixture: LocalHttpFixture) -> None:
840
841      # 当前示例仍拥有底层工作的释放事件，所以必须放行并确认线程结束。
842      may_finish.set()
843      await wait_for_thread_event(finished, "取消后仍运行的同步线程结束")
844      # list 在两个线程间共享；此处只在 finished 发信号后读取，观察底层工作最终追加的副作用。
845      print("   稍后观察到底层结果:", side_effects)
846      assert side_effects == ["线程中的底层工作仍然完成"]
847      print("4) CPU 密集工作需另选策略；to_thread 主要用于避免阻塞事件循环。")
848
849
850  # 组合示例把前面概念放进一个请求作用域：一个绝对 deadline 约束整组工作，
851  # HTTP 分支持有共享客户端/并发许可，SQLite 分支在线程中持有自己的连接。
852  async def demonstrate_combined(fixture: LocalHttpFixture) -> None:
853      print_section("组合示例：一个 deadline 下共享客户端、并发许可与数据库线程")
854      loop: AbstractEventLoop = asyncio.get_running_loop()
855      # 绝对截止时刻只计算一次，并同时传给外层 timeout_at 和内部 HTTP 重试状态机。
856      deadline: float = loop.time() + 1.0
857      # 组合场景允许最多两个同时在途的 HTTP 尝试；本例实际只有一个 HTTP Task。
858      limiter = asyncio.Semaphore(2)
859      trace: list[str] = []
860
861      # 上下文嵌套顺序也表达所有权：退出时先关最内层 AsyncClient，再清理最外层临时目录。
862      with tempfile.TemporaryDirectory(prefix="chapter25-combined-") as temp_directory:
863          database_path: Path = Path(temp_directory) / "documents.sqlite3"
864          await asyncio.to_thread(create_database, database_path)
865
866          # 本节单独给客户端两条连接容量；client 的生命周期覆盖整个 HTTP 子任务。
867          async with httpx.AsyncClient(
868              base_url=fixture.base_url,
869              timeout=HTTP_TIMEOUT,
870              limits=httpx.Limits(max_connections=2, max_keepalive_connections=2),
871              trust_env=False,
872          ) as client:
873              # timeout_at 约束 TaskGroup 的整体等待。TaskGroup 是结构化并发作用域：
874              # 离开前会等待所有子 Task；若一个子任务失败，会取消仍运行的兄弟并汇总异常。
875              async with asyncio.timeout_at(deadline):
```

```python
async def demonstrate_combined(fixture: LocalHttpFixture) -> None:
        # 本节单独给客户端两条连接容量；client 的生命周期覆盖整个 HTTP 子任务。
        async with httpx.AsyncClient(
            base_url=fixture.base_url,
            timeout=HTTP_TIMEOUT,
            limits=httpx.Limits(max_connections=2, max_keepalive_connections=2),
            trust_env=False,
        ) as client:
            # timeout_at 约束 TaskGroup 的整体等待。TaskGroup 是结构化并发作用域：
            # 离开前会等待所有子 Task；若一个子任务失败，会取消仍运行的兄弟并汇总异常。
            async with asyncio.timeout_at(deadline):
                async with asyncio.TaskGroup() as group:
                    # group.create_task 返回有类型结果的 Task，成功退出组后可用 result() 同步取值。
                    http_task: Task[Document] = group.create_task(
                        fetch_read_only_with_retry(
                            client,
                            "组合查询",
                            limiter=limiter,
                            deadline=deadline,
                            trace=trace,
                        ),
                        name="combined-http",
                    )
                    # 这里传给 to_thread 的仍是普通函数和实参；数据库连接会在该工作线程内新建。
                    database_task: Task[list[DatabaseDocument]] = group.create_task(
                        asyncio.to_thread(query_database, database_path, 0.90),
                        name="combined-database",
                    )

                # 能执行到这里说明 TaskGroup 已正常结束，两个 result() 都不会再阻塞。
                remaining: float = max(0.0, deadline - loop.time())
                print("HTTP 结果:", http_task.result().title)
                print("数据库结果数:", len(database_task.result()))
                print("HTTP 尝试记录:", trace)
                print(f"退出组合工作后剩余预算: {remaining:.3f}s")

        # 读输出行于开局资源上下文之外，表示客户端和监时局灵清理都已执行：
```

```
852  async def demonstrate_combined(fixture: LocalHttpFixture) -> None:
901      # 该输出位于两层资源上下文之外，表示客户端和临时目录清理都已执行；
902      # SQLite 连接更早已在各自的同步函数返回前关闭。
903      print("组合示例已关闭客户端、数据库连接和临时目录。")
904
905
906  # section 是命令行解析出的普通字符串。retry 完全基于 MockTransport，所以无需启动本地服务；
907  # 其他小节共享同一个 fixture，上下文退出时统一关闭监听 socket 和服务线程。
908  async def run_examples(section: str) -> None:
909      if section == "retry":
910          await demonstrate_retries()
911          # 提前 return，避免为 retry 分支创建从未使用的本地 HTTP 服务。
912          return
913
914      # LocalHttpFixture 是同步上下文管理器，因为其启动/关闭接口本身不是协程；
915      # 内部服务跑在线程中；当前 asyncio 事件循环负责客户端侧实验。
916      with LocalHttpFixture() as fixture:
917          # 集合成员测试把 all 与单节入口合并，避免重复写调用代码。
918          if section in {"all", "http"}:
919              await demonstrate_httpx(fixture)
920          if section == "all":
921              await demonstrate_retries()
922          if section in {"all", "sync"}:
923              await demonstrate_sync_io(fixture)
924          if section in {"all", "combined"}:
925              await demonstrate_combined(fixture)
926
927      # with 已退出但 fixture 变量仍可引用对象，因此可以读取 closed 标志验证清理状态。
928      print("\n本地 HTTP 服务已关闭:", fixture.closed)
929
930
931  # argparse 把命令行文本转换成 Namespace；这里只定义参数规则，不运行任何异步代码。
932  def parse_args() -> argparse.Namespace:
933      # 模块 docstring 作为 `--help` 的说明文本；这是文件原本已有的真实 __doc__。
934      parser = argparse.ArgumentParser(description=__doc__)
935      parser.add_argument(
```

```
929
930
931 # argparse 把命令行文本转换成 Namespace；这里只定义参数规则，不运行任何异步代码。
932 def parse_args() -> argparse.Namespace:
933     # 模块 docstring 作为 `--help` 的说明文本；这是文件原本已有的真实 __doc__。
934     parser = argparse.ArgumentParser(description=__doc__)
935     parser.add_argument(
936         "section",
937         # nargs="?" 表示位置参数可出现 0 或 1 次；省略时使用 default="all"。
938         nargs="?",
939         # choices 在进入业务逻辑前拒绝其他字符串，所以 run_examples 只需处理这五种值。
940         choices=("all", "http", "retry", "sync", "combined"),
941         default="all",
942         help="选择单节运行；默认 all",
943     )
944     return parser.parse_args()
945
946
947 # 同步 main 负责解析 CLI，并用 asyncio.run 创建事件循环、运行顶层协程、最后关闭循环。
948 # asyncio.run 应位于程序边界；已经运行在事件循环中的库代码通常应直接 await run_examples。
949 def main() -> None:
950     arguments: Namespace = parse_args()
951     asyncio.run(run_examples(arguments.section))
952
953
954 # Python 导入该文件时只定义上面的类和函数，不执行示例；直接运行脚本时 __name__ 才是
955 # "__main__"，于是调用 main。作用近似 Java 的程序入口约定，但由模块运行方式决定。
956 if __name__ == "__main__":
957     main()
958
```

```python
"""第 25 章的本地 HTTP 服务。

客户端代码才是本章重点。这个 fixture 只负责制造真实的回环 TCP 传输、可控慢响应、
HTTP/1.1 连接复用和 JSONL 流，并确保所有等待都有清理上限。
"""

import json
import sys
import threading
from dataclasses import dataclass, field
from http import HTTPStatus
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from socket import socket
from typing import Self, cast
from urllib.parse import parse_qs, urlsplit


@dataclass(frozen=True, slots=True)
class RequestRecord:
    """服务端观察到的一次请求；客户端端口可辅助观察 TCP 连接复用。"""

    path: str
    client_port: int


@dataclass(slots=True)
class FixtureState:
    """由事件协调客户端与服务线程，不靠 sleep 猜先后顺序。"""

    slow_request_started: threading.Event = field(default_factory=threading.Event)
    release_slow_response: threading.Event = field(default_factory=threading.Event)
    slow_request_finished: threading.Event = field(default_factory=threading.Event)

    first_stream_line_sent: threading.Event = field(default_factory=threading.Event)
    release_stream_tail: threading.Event = field(default_factory=threading.Event)
    stream_request_finished: threading.Event = field(default_factory=threading.Event)

```

```python
class FixtureState:

    sync_request_started: threading.Event = field(default_factory=threading.Event)
    event_loop_heartbeat: threading.Event = field(default_factory=threading.Event)

    server_thread_started: threading.Event = field(default_factory=threading.Event)
    _records: list[RequestRecord] = field(default_factory=list)
    _records_lock: threading.Lock = field(default_factory=threading.Lock)

    def record(self, path: str, client_port: int) -> None:
        with self._records_lock:
            self._records.append(RequestRecord(path, client_port))

    def records_snapshot(self) -> list[RequestRecord]:
        with self._records_lock:
            return list(self._records)

    def reset_slow_exchange(self) -> None:
        """前一轮服务线程结束后，给下一个慢请求使用一组未触发事件。"""
        if self.slow_request_started.is_set() and not self.slow_request_finished.is_set():
            raise RuntimeError("上一轮慢请求尚未结束")
        self.slow_request_started.clear()
        self.release_slow_response.clear()
        self.slow_request_finished.clear()


class FixtureServer(ThreadingHTTPServer):
    """给标准库 HTTP 服务器附加测试状态。"""

    daemon_threads = True

    def __init__(
        self,
        server_address: tuple[str, int],
        handler_class: type[BaseHTTPRequestHandler],
        state: FixtureState,
    ) -> None:
```

```python
class FixtureServer(ThreadingHTTPServer):

    def __init__(
        self,
        server_address: tuple[str, int],
        handler_class: type[BaseHTTPRequestHandler],
        state: FixtureState,
    ) -> None:
        self.state: FixtureState = state
        super().__init__(server_address, handler_class)

    def handle_error(
        self,
        request: socket | tuple[bytes, socket],
        client_address: object,
    ) -> None:
        """提前关闭流/超时会重置测试连接；这是预期路径，不打印服务端 traceback。"""
        error: BaseException | None = sys.exception()
        if isinstance(error, (BrokenPipeError, ConnectionResetError)):
            return
        super().handle_error(request, client_address)


class FixtureRequestHandler(BaseHTTPRequestHandler):
    """提供固定响应；不属于需要学习的生产服务端实现。"""

    protocol_version = "HTTP/1.1"

    @property
    def fixture_server(self) -> FixtureServer:
        if not isinstance(self.server, FixtureServer):
            raise TypeError("fixture handler 必须由 FixtureServer 承载")
        return self.server

    def log_message(self, format_string: str, *args: object) -> None:
        """关闭并发服务线程日志，保持教学输出顺序可读。"""

```

```
    class FixtureRequestHandler(BaseHTTPRequestHandler):
        def do_GET(self) -> None:
            state: FixtureState = self.fixture_server.state
            target: SplitResult = urlsplit(self.path)
            state.record(target.path, self.client_address[1])

            if target.path == "/document":
                query: str = parse_qs(target.query).get("query", ["Agent"])[0]
                self._send_json(
                    HTTPStatus.OK,
                    {"id": 25, "title": f"{query} 可靠 I/O", "score": 0.95},
                )
                return
            if target.path == "/slow":
                self._send_slow_response()
                return
            if target.path == "/events":
                self._send_jsonl_stream()
                return
            if target.path == "/sync-wait":
                self._wait_for_event_loop_heartbeat()
                return
            self._send_json(
                HTTPStatus.NOT_FOUND,
                {"code": "NOT_FOUND", "message": "本地 fixture 没有这个端点"},
            )

        def _send_slow_response(self) -> None:
            state: FixtureState = self.fixture_server.state
            state.slow_request_started.set()
            # 这是清理保险上限。正常实验会在客户端 finally 中显式放行。
            released: bool = state.release_slow_response.wait(timeout=2.0)
            try:
                if released:
                    self._send_json(HTTPStatus.OK, {"message": "慢响应已放行"})
```

```
 88  class FixtureRequestHandler(BaseHTTPRequestHandler):
129      def _send_slow_response(self) -> None:
134          try:
135              if released:
136                  self._send_json(HTTPStatus.OK, {"message": "慢响应已放行"})
137          finally:
138              state.slow_request_finished.set()
139
140      def _send_jsonl_stream(self) -> None:
141          state: FixtureState = self.fixture_server.state
142          self.send_response(HTTPStatus.OK)
143          self.send_header("Content-Type", "application/x-ndjson; charset=utf-8")
144          self.send_header("Transfer-Encoding", "chunked")
145          self.end_headers()
146
147          try:
148              self._write_chunk({"type": "token", "text": "你好"})
149              state.first_stream_line_sent.set()
150              # 客户端读到第一行并退出 stream 上下文后才放行后续写入。
151              if state.release_stream_tail.wait(timeout=2.0):
152                  self._write_chunk({"type": "done", "usage": 1})
153                  self.wfile.write(b"0\r\n\r\n")
154                  self.wfile.flush()
155          except BrokenPipeError, ConnectionResetError:
156              # 提前停止流时，客户端关闭连接后服务端继续写入失败是预期路径。
157              pass
158          finally:
159              state.stream_request_finished.set()
160
161      def _write_chunk(self, payload: object) -> None:
162          line: bytes = (json.dumps(payload, ensure_ascii=False) + "\n").encode()
163          self.wfile.write(f"{len(line):X}\r\n".encode("ascii"))
164          self.wfile.write(line)
165          self.wfile.write(b"\r\n")
166          self.wfile.flush()
167
```

```
 88  class FixtureRequestHandler(BaseHTTPRequestHandler):
167
168      def _wait_for_event_loop_heartbeat(self) -> None:
169          state: FixtureState = self.fixture_server.state
170          state.sync_request_started.set()
171          # 同步客户端若堵住事件循环，计划中的 heartbeat 无法及时运行。
172          heartbeat_seen: bool = state.event_loop_heartbeat.wait(timeout=0.25)
173          self._send_json(HTTPStatus.OK, {"heartbeat_seen": heartbeat_seen})
174
175      def _send_json(self, status: HTTPStatus, payload: object) -> None:
176          body: bytes = json.dumps(payload, ensure_ascii=False).encode()
177          self.send_response(status)
178          self.send_header("Content-Type", "application/json; charset=utf-8")
179          self.send_header("Content-Length", str(len(body)))
180          self.end_headers()
181          try:
182              self.wfile.write(body)
183              self.wfile.flush()
184          except BrokenPipeError, ConnectionResetError:
185              # read timeout 后客户端可能已关闭连接。
186              pass
187
188
189  class LocalHttpFixture:
190      """自动选择端口并拥有服务线程的同步上下文管理器。"""
191
192      def __init__(self) -> None:
193          self.state = FixtureState()
194          self.base_url: Literal[''] = ""
195          self.closed = False
196          self._server: FixtureServer | None = None
197          self._thread: threading.Thread | None = None
198
199      def __enter__(self) -> Self:
200          server = FixtureServer(("127.0.0.1", 0), FixtureRequestHandler, self.state)
201          self._server = server
```

```python
class LocalHttpFixture:

    def __enter__(self) -> Self:
        server = FixtureServer(("127.0.0.1", 0), FixtureRequestHandler, self.state)
        self._server = server
        host: str, port: int = cast(tuple[str, int], server.server_address)
        self.base_url: str = f"http://{host}:{port}"

        def serve() -> None:
            self.state.server_thread_started.set()
            server.serve_forever(poll_interval=0.02)

        thread = threading.Thread(target=serve, name="chapter25-http-fixture", daemon=True)
        self._thread = thread
        thread.start()
        if not self.state.server_thread_started.wait(timeout=1.0):
            self.__exit__(None, None, None)
            raise RuntimeError("本地 HTTP fixture 未能启动")
        return self

    def __exit__(self, exc_type: object, exc_value: object, traceback: object) -> None:
        # 先打开全部闸门，异常路径也不会遗留永久等待的服务线程。
        self.state.release_slow_response.set()
        self.state.release_stream_tail.set()
        self.state.event_loop_heartbeat.set()
        if self._server is not None:
            self._server.shutdown()
            self._server.server_close()
        if self._thread is not None:
            self._thread.join(timeout=2.0)
            if self._thread.is_alive():
                raise RuntimeError("本地 HTTP fixture 服务线程没有按期退出")
        self.closed = True

```

```
1  httpx==0.28.1
2  pydantic==2.13.5
3  requests==2.34.2
4
```